

File Tree

해시 확인: certutil -hashfile <dist> SHA256

dist/

```
├─ assets/
│   └─ index-av5gkzHq.css (11844 B / 017d5c46ea47)
└─ index-CkN8R2zc.js (98313 B / 91c6a85e0695)
└─ index.html (6983 B / a6648d94f2cc)
```

==== assets/index-av5gkzHq.css (11844 B / 017d5c46ea47) ====

```
1 | *,
2 | *:before,
3 | *:after {
4 |   box-sizing: border-box;
5 | }
6 | html {
7 |   min-width: 1280px;
8 |   color-scheme: light;
9 | }
10 | body {
11 |   min-width: 1280px;
12 |   min-height: 720px;
13 |   margin: 0;
14 |   font-family:
15 |     Inter,
16 |     ui-sans-serif,
17 |     system-ui,
18 |     -apple-system,
19 |     BlinkMacSystemFont,
20 |     Segoe UI,
21 |     sans-serif;
22 |   text-rendering: geometricPrecision;
23 | }
24 | button {
25 |   font: inherit;
26 | }
27 | button:not(:disabled) {
28 |   cursor: pointer;
29 | }
30 | button:disabled {
31 |   cursor: not-allowed;
32 | }
33 | :root {
34 |   --app-padding: 28px;
35 |   --page: #f3eee3;
36 |   --ink: #20252b;
37 |   --muted: #66716f;
38 |   --line: #c9c0b1;
39 |   --panel: #fffaf0;
40 |   --button: #263a3c;
41 |   --button-text: #ffffff;
42 |   --accent: #b94b36;
43 |   --light-square: #eadbb7;
44 |   --dark-square: #5f8a65;
45 |   --last-move: #e0b84b;
46 |   --legal: #264f8f;
47 |   --capture: #b83232;
48 |   --shadow: 0 18px 42px rgb(36 31 24 / 18%);
49 | }
50 | [hidden] {
51 |   display: none !important;
52 | }
53 | .chess-app {
54 |   --board-max-size: 560px;
55 |   width: 100%;
56 |   min-width: 1280px;
57 |   height: 720px;
58 |   min-height: 720px;
59 |   display: grid;
60 |   grid-template-columns: 230px var(--board-max-size) 118px 280px;
```

```

61 | gap: 10px;
62 | align-items: stretch;
63 | justify-content: center;
64 | overflow: hidden;
65 | padding: var(--app-padding);
66 | background: var(--page);
67 | color: var(--ink);
68 | }
69 | .board-stage {
70 |   width: var(--board-max-size);
71 |   align-self: center;
72 |   justify-self: center;
73 | }
74 | .clock-panel {
75 |   width: 100%;
76 |   height: var(--board-max-size);
77 |   min-width: 0;
78 |   min-height: 0;
79 |   align-self: center;
80 |   justify-self: stretch;
81 |   display: flex;
82 | }
83 | .chess-board {
84 |   width: 100%;
85 |   aspect-ratio: 1;
86 |   display: grid;
87 |   grid-template-columns: repeat(8, minmax(0, 1fr));
88 |   overflow: hidden;
89 |   border: 3px solid #2e3a32;
90 |   border-radius: 8px;
91 |   box-shadow: var(--shadow);
92 |   background: #2e3a32;
93 | }
94 | .board-square {
95 |   position: relative;
96 |   width: 100%;
97 |   aspect-ratio: 1;
98 |   display: grid;
99 |   place-items: center;
100 |   min-width: 0;
101 |   padding: 0;
102 |   border: 0;
103 |   color: var(--ink);
104 |   transition:
105 |     box-shadow 0.14s ease,
106 |     filter 0.14s ease,
107 |     transform 0.14s ease;
108 | }
109 | .board-square:not(:disabled):hover {
110 |   filter: brightness(1.05);
111 | }
112 | .board-square.is-light {
113 |   background: var(--light-square);
114 | }
115 | .board-square.is-dark {
116 |   background: var(--dark-square);
117 | }
118 | .board-square.is-selected {
119 |   box-shadow: inset 0 0 4px var(--legal);
120 | }
121 | .board-square.is-last-move {
122 |   box-shadow: inset 0 0 4px #e0b84bc2;
123 | }
124 | .board-square.is-selected.is-last-move {
125 |   box-shadow:
126 |     inset 0 0 4px var(--legal),
127 |     inset 0 0 8px #e0b84bad;
128 | }
129 | .board-square.is-legal:after,
130 | .board-square.is-capture:after {
131 |   position: absolute;

```

```

132 |   z-index: 1;
133 |   content: '';
134 |   border-radius: 999px;
135 |   pointer-events: none;
136 | }
137 | .board-square.is-legal:after {
138 |   width: 28%;
139 |   height: 28%;
140 |   background: #264f8f6b;
141 | }
142 | .board-square.is-capture:after {
143 |   inset: 12%;
144 |   border: 3px solid rgba(184, 50, 50, 0.74);
145 | }
146 | .board-square.is-check {
147 |   box-shadow: inset 0 0 0.5px var(--capture);
148 | }
149 | .chess-board.is-locked .board-square {
150 |   filter: saturate(0.86);
151 | }
152 | .piece {
153 |   z-index: 2;
154 |   display: block;
155 |   font-size: 4rem;
156 |   line-height: 1;
157 |   text-shadow: 0 2px 6px rgba(0, 0, 0, 0.2);
158 |   -webkit-user-select: none;
159 |   user-select: none;
160 | }
161 | .coordinate {
162 |   position: absolute;
163 |   z-index: 3;
164 |   font-size: 0.7rem;
165 |   font-weight: 800;
166 |   line-height: 1;
167 |   opacity: 0.72;
168 |   pointer-events: none;
169 | }
170 | .coordinate-rank {
171 |   top: 6px;
172 |   left: 7px;
173 | }
174 | .coordinate-file {
175 |   right: 7px;
176 |   bottom: 6px;
177 | }
178 | .control-panel,
179 | .info-panel {
180 |   min-width: 0;
181 |   min-height: 0;
182 |   max-height: 664px;
183 |   display: flex;
184 |   flex-direction: column;
185 |   gap: 14px;
186 |   overflow: auto;
187 |   overscroll-behavior: contain;
188 |   scrollbar-gutter: stable;
189 | }
190 | .control-panel {
191 |   justify-self: end;
192 | }
193 | .info-panel {
194 |   justify-self: start;
195 | }
196 | .control-label {
197 |   margin: 0;
198 |   color: var(--muted);
199 |   font-size: 0.78rem;
200 |   font-weight: 800;
201 |   letter-spacing: 0;
202 |   text-transform: uppercase;

```

```

203 | }
204 | .control-group,
205 | .status-panel,
206 | .ai-candidate-panel,
207 | .history-panel-{
208 |   display: grid;
209 |   gap: 10px;
210 |   padding-top: 16px;
211 |   border-top: 1px solid var(--line);
212 | }
213 | .history-panel-{
214 |   min-height: 0;
215 |   flex: 1;
216 |   grid-template-rows: auto minmax(0, 1fr);
217 | }
218 | .segmented-control-{
219 |   display: grid;
220 |   grid-template-columns: repeat(2, minmax(0, 1fr));
221 |   gap: 4px;
222 |   padding: 4px;
223 |   border: 1px solid var(--line);
224 |   border-radius: 8px;
225 |   background: #fffaf0ad;
226 | }
227 | .segmented-control-three-{
228 |   grid-template-columns: repeat(3, minmax(0, 1fr));
229 | }
230 | .clock-preset-control-{
231 |   display: grid;
232 |   grid-template-columns: repeat(5, minmax(0, 1fr));
233 |   gap: 4px;
234 |   padding: 4px;
235 |   border: 1px solid var(--line);
236 |   border-radius: 8px;
237 |   background: #fffaf0ad;
238 | }
239 | .segmented-button,
240 | .primary-button,
241 | .ghost-button,
242 | .promotion-button-{
243 |   min-height: 42px;
244 |   border-radius: 6px;
245 |   font-weight: 800;
246 |   letter-spacing: 0;
247 | }
248 | .segmented-button-{
249 |   border: 0;
250 |   background: transparent;
251 |   color: var(--muted);
252 | }
253 | .segmented-button.is-active-{
254 |   background: var(--button);
255 |   color: var(--button-text);
256 | }
257 | .clock-settings .segmented-button-{
258 |   min-height: 36px;
259 |   padding-inline: 4px;
260 |   font-size: 0.82rem;
261 | }
262 | .ai-settings-{
263 |   gap: 12px;
264 | }
265 | .range-control,
266 | .toggle-control-{
267 |   display: grid;
268 |   gap: 8px;
269 |   min-width: 0;
270 |   color: var(--ink);
271 |   font-size: 0.92rem;
272 |   font-weight: 800;
273 | }

```

```

274 | .range-control-span,
275 | .toggle-control-{
276 | | align-items: center;
277 | }
278 | .range-control-span-{
279 | | display: flex;
280 | | justify-content: space-between;
281 | | gap: 10px;
282 | }
283 | .range-control-strong-{
284 | | color: var(--accent);
285 | | white-space: nowrap;
286 | }
287 | .range-control-input[type='range']-{
288 | | width: 100%;
289 | | accent-color: var(--button);
290 | }
291 | .toggle-control-{
292 | | grid-template-columns: auto minmax(0, 1fr);
293 | | padding: 10px;
294 | | border: 1px solid var(--line);
295 | | border-radius: 8px;
296 | | background: #fffaf0bd;
297 | }
298 | .toggle-control-input-{
299 | | width: 18px;
300 | | height: 18px;
301 | | accent-color: var(--button);
302 | }
303 | .action-row-{
304 | | display: grid;
305 | | grid-template-columns: 1fr 1fr;
306 | | gap: 10px;
307 | }
308 | .primary-button,
309 | .ghost-button-{
310 | | border: 1px solid var(--button);
311 | }
312 | .primary-button-{
313 | | background: var(--button);
314 | | color: var(--button-text);
315 | }
316 | .ghost-button-{
317 | | background: transparent;
318 | | color: var(--button);
319 | }
320 | .primary-button:disabled,
321 | .ghost-button:disabled,
322 | .segmented-button:disabled-{
323 | | opacity: 0.46;
324 | }
325 | .game-status-{
326 | | min-height: 1.4em;
327 | | color: var(--accent);
328 | | font-size: 1.35rem;
329 | | line-height: 1.2;
330 | }
331 | .clock-display-{
332 | | width: 100%;
333 | | height: 100%;
334 | | min-height: 0;
335 | | min-width: 0;
336 | | display: flex;
337 | | flex-direction: column;
338 | | gap: 10px;
339 | | justify-content: space-between;
340 | }
341 | .clock-card-{
342 | | --clock-bg: rgb(255 250 240 / 74%);
343 | | --clock-border: var(--line);
344 | | --clock-side: var(--muted);

```

```

345 | --clock-time: var(--ink);
346 | width: 100%;
347 | min-block-size: 82px;
348 | max-block-size: 110px;
349 | min-width: 0;
350 | display: grid;
351 | place-content: center;
352 | gap: 4px;
353 | padding: 12px 8px;
354 | border: 1px solid var(--clock-border);
355 | border-radius: 8px;
356 | background: var(--clock-bg);
357 | text-align: center;
358 | }
359 | #white-clock {
360 |   --clock-bg: #fffaf0;
361 |   --clock-border: #c9c0b1;
362 |   --clock-side: #66716f;
363 |   --clock-time: #20252b;
364 | }
365 | #black-clock {
366 |   --clock-bg: #263a3c;
367 |   --clock-border: #172426;
368 |   --clock-side: rgb(255 250 240 / 74%);
369 |   --clock-time: #fffaf0;
370 | }
371 | .clock-card.is-active {
372 |   border-color: var(--button);
373 |   box-shadow:
374 |     inset 0 0 0 2px #ffffff4d,
375 |     0 0 0 2px #263a3c2e;
376 | }
377 | .clock-card.is-warning .clock-time {
378 |   color: #d18b00;
379 | }
380 | .clock-card.is-danger .clock-time,
381 | .clock-card.is-timeout .clock-time {
382 |   color: var(--capture);
383 | }
384 | .clock-side {
385 |   color: var(--clock-side);
386 |   font-size: 0.72rem;
387 |   font-weight: 800;
388 |   line-height: 1;
389 | }
390 | .clock-time {
391 |   font-variant-numeric: tabular-nums;
392 |   color: var(--clock-time);
393 |   font-size: 1.35rem;
394 |   line-height: 1;
395 | }
396 | .status-grid {
397 |   display: grid;
398 |   grid-template-columns: 1fr;
399 |   gap: 8px;
400 |   margin: 0;
401 | }
402 | .status-grid div {
403 |   min-width: 0;
404 |   padding: 10px;
405 |   border: 1px solid var(--line);
406 |   border-radius: 8px;
407 |   background: #fffaf0bd;
408 | }
409 | .status-grid dt,
410 | .status-grid dd {
411 |   margin: 0;
412 | }
413 | .status-grid dt {
414 |   color: var(--muted);
415 |   font-size: 0.72rem;

```

```

416 |   font-weight: 800;
417 | }
418 | .status-grid-dd {
419 |   margin-top: 4px;
420 |   overflow-wrap: anywhere;
421 |   font-size: 0.92rem;
422 |   font-weight: 800;
423 | }
424 | .history-header {
425 |   display: flex;
426 |   align-items: center;
427 |   justify-content: space-between;
428 | }
429 | .ai-candidate-panel {
430 |   min-height: 0;
431 | }
432 | .ai-info-grid {
433 |   display: grid;
434 |   grid-template-columns: repeat(2, minmax(0, 1fr));
435 |   gap: 6px;
436 |   margin: 0;
437 | }
438 | .ai-info-grid-div {
439 |   min-width: 0;
440 |   padding: 8px;
441 |   border: 1px solid var(--line);
442 |   border-radius: 8px;
443 |   background: #fffaf0bd;
444 | }
445 | .ai-info-grid-dt,
446 | .ai-info-grid-dd {
447 |   margin: 0;
448 | }
449 | .ai-info-grid-dt {
450 |   color: var(--muted);
451 |   font-size: 0.64rem;
452 |   font-weight: 800;
453 |   text-transform: uppercase;
454 | }
455 | .ai-info-grid-dd {
456 |   margin-top: 3px;
457 |   overflow: hidden;
458 |   font-size: 0.8rem;
459 |   font-weight: 800;
460 |   line-height: 1.1;
461 |   text-overflow: ellipsis;
462 |   white-space: nowrap;
463 | }
464 | .ai-info-grid-dd[id='ai-info-nodes'],
465 | .ai-info-grid-dd[id='ai-info-score'],
466 | .ai-info-grid-dd[id='ai-info-depth'] {
467 |   font-variant-numeric: tabular-nums;
468 | }
469 | .ai-candidate-header {
470 |   display: flex;
471 |   align-items: baseline;
472 |   justify-content: space-between;
473 |   gap: 8px;
474 | }
475 | .ai-candidate-meta {
476 |   min-width: 0;
477 |   overflow: hidden;
478 |   color: var(--muted);
479 |   font-size: 0.68rem;
480 |   font-weight: 800;
481 |   text-align: right;
482 |   text-overflow: ellipsis;
483 |   white-space: nowrap;
484 | }
485 | .ai-candidate-table {
486 |   min-height: 0;

```

```
487 | overflow: hidden;
488 | border: 1px solid var(--line);
489 | border-radius: 8px;
490 | background: #fffaf0bd;
491 | }
492 | .ai-candidate-list {
493 |   max-height: 172px;
494 |   overflow: auto;
495 | }
496 | .ai-candidate-row {
497 |   display: grid;
498 |   grid-template-columns: minmax(3rem, 1.3fr) 2.6rem 3.2rem 3.2rem;
499 |   gap: 6px;
500 |   align-items: center;
501 |   min-width: 0;
502 |   padding: 6px 8px;
503 |   border-bottom: 1px solid rgba(201, 192, 177, 0.58);
504 |   font-size: 0.72rem;
505 |   font-weight: 800;
506 |   line-height: 1.15;
507 | }
508 | .ai-candidate-row:last-child {
509 |   border-bottom: 0;
510 | }
511 | .ai-candidate-head {
512 |   color: var(--muted);
513 |   background: #fffaf0e0;
514 |   font-size: 0.66rem;
515 |   text-transform: uppercase;
516 | }
517 | .ai-candidate-row.is-selected {
518 |   background: #b94b361f;
519 | }
520 | .ai-candidate-row.is-principal:not(.is-selected) {
521 |   background: #263a3c14;
522 | }
523 | .ai-candidate-san,
524 | .ai-candidate-score,
525 | .ai-candidate-weight,
526 | .ai-candidate-probability {
527 |   min-width: 0;
528 |   overflow: hidden;
529 |   text-overflow: ellipsis;
530 |   white-space: nowrap;
531 | }
532 | .ai-candidate-score,
533 | .ai-candidate-weight,
534 | .ai-candidate-probability {
535 |   text-align: right;
536 |   font-variant-numeric: tabular-nums;
537 | }
538 | .ai-candidate-badge {
539 |   display: inline-block;
540 |   margin-left: 4px;
541 |   padding: 1px 3px;
542 |   border-radius: 4px;
543 |   background: var(--button);
544 |   color: var(--button-text);
545 |   font-size: 0.56rem;
546 |   line-height: 1;
547 |   vertical-align: 1px;
548 | }
549 | .move-list {
550 |   min-height: 0;
551 |   max-height: none;
552 |   overflow: auto;
553 |   border: 1px solid var(--line);
554 |   border-radius: 8px;
555 |   background: #fffaf0bd;
556 | }
557 | .move-row {
```



```

558 |   display: grid;
559 |   grid-template-columns: 3rem minmax(0, 1fr) minmax(0, 1fr);
560 |   gap: 8px;
561 |   padding: 8px 10px;
562 |   border-bottom: 1px solid rgba(201, 192, 177, 0.62);
563 |   font-weight: 800;
564 | }
565 | .move-row:last-child {
566 |   border-bottom: 0;
567 | }
568 | .move-number {
569 |   color: var(--muted);
570 | }
571 | .move-san {
572 |   min-width: 0;
573 |   overflow-wrap: anywhere;
574 | }
575 | .move-empty {
576 |   margin: 0;
577 |   padding: 14px 12px;
578 |   color: var(--muted);
579 |   font-weight: 700;
580 | }
581 | .promotion-overlay {
582 |   position: fixed;
583 |   inset: 0;
584 |   z-index: 10;
585 |   display: grid;
586 |   place-items: center;
587 |   padding: 20px;
588 |   background: #15191c94;
589 | }
590 | .promotion-overlay[hidden] {
591 |   display: none;
592 | }
593 | .promotion-dialog {
594 |   width: 360px;
595 |   display: grid;
596 |   gap: 16px;
597 |   padding: 20px;
598 |   border: 1px solid var(--line);
599 |   border-radius: 8px;
600 |   background: var(--panel);
601 |   box-shadow: var(--shadow);
602 | }
603 | .promotion-dialog h2 {
604 |   margin: 0;
605 |   font-size: 1.1rem;
606 | }
607 | .promotion-choices {
608 |   display: grid;
609 |   grid-template-columns: repeat(4, minmax(0, 1fr));
610 |   gap: 8px;
611 | }
612 | .promotion-button {
613 |   min-height: 70px;
614 |   border: 1px solid var(--button);
615 |   background: #fff;
616 |   color: var(--ink);
617 |   font-size: 2.4rem;
618 |   line-height: 1;
619 | }
620 |

```

==== assets/index-CkN8R2zc.js (98313 B / 91c6a85e0695) ====

```

1 | const MASK64 = 0xffffffffffffffff;
2 | function rotl(x, k) {
3 |   return 0xffffffffffffffff & ((x << k) | (x >> (64n - k)));
4 | }
5 | function wrappingMul(x, y) {
6 |   return (x * y) & MASK64;

```

```

7 | }
8 | const rand = (function xoroshiro128(state) {
9 |     return function () {
10 |         let s0 = BigInt(state & MASK64),
11 |            s1 = BigInt((state >> 64n) & MASK64);
12 |         const result = wrappingMul(rotl(wrappingMul(s0, 5n), 7n), 9n);
13 |         return (
14 |             (s1 ^ s0),
15 |             (s0 = (rotl(s0, 24n) ^ s1 ^ (s1 << 16n)) & MASK64),
16 |             (s1 = rotl(s1, 37n)),
17 |             (state = (s1 << 64n) | s0),
18 |             result
19 |         );
20 |     };
21 | })(0xa187eb39cdcaed8f31c4b365b102e01en),
22 | PIECE_KEYS = Array.from({ length: 2 }, () =>
23 |     Array.from({ length: 6 }, () => Array.from({ length: 128 }, () => rand()))),
24 | ),
25 | EP_KEYS = Array.from({ length: 8 }, () => rand()),
26 | CASTLING_KEYS = Array.from({ length: 16 }, () => rand()),
27 | SIDE_KEY = rand(),
28 | DEFAULT_POSITION = 'rnbqkbnr/pppppppp/8/8/8/PPPPPPPP/RNBQKBNR.w-KQkq--0-1';
29 | class Move {
30 |     color;
31 |     from;
32 |     to;
33 |     piece;
34 |     captured;
35 |     promotion;
36 |     flags;
37 |     san;
38 |     lan;
39 |     before;
40 |     after;
41 |     constructor(chess, internal) {
42 |         const {
43 |             color: color,
44 |             piece: piece,
45 |             from: from,
46 |             to: to,
47 |             flags: flags,
48 |             captured: captured,
49 |             promotion: promotion,
50 |         } = internal,
51 |             fromAlgebraic = algebraic(from),
52 |             toAlgebraic = algebraic(to);
53 |         ((this.color = color),
54 |         (this.piece = piece),
55 |         (this.from = fromAlgebraic),
56 |         (this.to = toAlgebraic),
57 |         (this.san = chess._moveToSan(internal, chess._moves({ legal: !0 }))),
58 |         (this.lan = fromAlgebraic + toAlgebraic),
59 |         (this.before = chess.fen()),
60 |         chess._makeMove(internal),
61 |         (this.after = chess.fen()),
62 |         chess._undoMove(),
63 |         (this.flags = ''));
64 |         for (const flag in BITS) BITS[flag] & flags && (this.flags += FLAGS[flag]);
65 |         (captured && (this.captured = captured),
66 |         promotion && ((this.promotion = promotion), (this.lan += promotion)));
67 |     }
68 |     isCapture() {
69 |         return this.flags.indexOf(FLAGS.CAPTURE) > -1;
70 |     }
71 |     isPromotion() {
72 |         return this.flags.indexOf(FLAGS.PROMOTION) > -1;
73 |     }
74 |     isEnPassant() {
75 |         return this.flags.indexOf(FLAGS.EP_CAPTURE) > -1;
76 |     }
77 |     isKingsideCastle() {

```

```

78 |     return this.flags.indexOf(FLAGS.KSIDE_CASTLE) > -1;
79 | }
80 | isQueensideCastle() {
81 |     return this.flags.indexOf(FLAGS.QSIDE_CASTLE) > -1;
82 | }
83 | isBigPawn() {
84 |     return this.flags.indexOf(FLAGS.BIG_PAWN) > -1;
85 | }
86 | }
87 | const FLAGS = {
88 |     NORMAL: 'n',
89 |     CAPTURE: 'c',
90 |     BIG_PAWN: 'b',
91 |     EP_CAPTURE: 'e',
92 |     PROMOTION: 'p',
93 |     KSIDE_CASTLE: 'k',
94 |     QSIDE_CASTLE: 'q',
95 |     NULL_MOVE: '-',
96 | },
97 | BITS = {
98 |     NORMAL: 1,
99 |     CAPTURE: 2,
100 |     BIG_PAWN: 4,
101 |     EP_CAPTURE: 8,
102 |     PROMOTION: 16,
103 |     KSIDE_CASTLE: 32,
104 |     QSIDE_CASTLE: 64,
105 |     NULL_MOVE: 128,
106 | },
107 | 0x88 = {
108 |     a8: 0,
109 |     b8: 1,
110 |     c8: 2,
111 |     d8: 3,
112 |     e8: 4,
113 |     f8: 5,
114 |     g8: 6,
115 |     h8: 7,
116 |     a7: 16,
117 |     b7: 17,
118 |     c7: 18,
119 |     d7: 19,
120 |     e7: 20,
121 |     f7: 21,
122 |     g7: 22,
123 |     h7: 23,
124 |     a6: 32,
125 |     b6: 33,
126 |     c6: 34,
127 |     d6: 35,
128 |     e6: 36,
129 |     f6: 37,
130 |     g6: 38,
131 |     h6: 39,
132 |     a5: 48,
133 |     b5: 49,
134 |     c5: 50,
135 |     d5: 51,
136 |     e5: 52,
137 |     f5: 53,
138 |     g5: 54,
139 |     h5: 55,
140 |     a4: 64,
141 |     b4: 65,
142 |     c4: 66,
143 |     d4: 67,
144 |     e4: 68,
145 |     f4: 69,
146 |     g4: 70,
147 |     h4: 71,
148 |     a3: 80,

```

```

149 |     b3: 81,
150 |     c3: 82,
151 |     d3: 83,
152 |     e3: 84,
153 |     f3: 85,
154 |     g3: 86,
155 |     h3: 87,
156 |     a2: 96,
157 |     b2: 97,
158 |     c2: 98,
159 |     d2: 99,
160 |     e2: 100,
161 |     f2: 101,
162 |     g2: 102,
163 |     h2: 103,
164 |     a1: 112,
165 |     b1: 113,
166 |     c1: 114,
167 |     d1: 115,
168 |     e1: 116,
169 |     f1: 117,
170 |     g1: 118,
171 |     h1: 119,
172 | },
173 | PAWN_OFFSETS = { b: [16, 32, 17, 15], w: [-16, -32, -17, -15] },
174 | PIECE_OFFSETS = {
175 |     n: [-18, -33, -31, -14, 18, 33, 31, 14],
176 |     b: [-17, -15, 17, 15],
177 |     r: [-16, 1, 16, -1],
178 |     q: [-17, -16, -15, 1, 17, 16, 15, -1],
179 |     k: [-17, -16, -15, 1, 17, 16, 15, -1],
180 | },
181 | ATTACKS = [
182 |     20, 0, 0, 0, 0, 0, 0, 0, 24, 0, 0, 0, 0, 0, 0, 0, 20, 0, 0, 0, 0, 0, 0, 0, 24, 0, 0, 0, 0, 0, 0, 24, 0, 0, 0, 0, 0, 20,
183 |     0, 0, 0, 0, 20, 0, 0, 0, 0, 24, 0, 0, 0, 0, 20, 0, 0, 0, 0, 0, 0, 20, 0, 0, 0, 0, 24, 0, 0, 0, 0, 24, 0, 0, 0, 0, 20,
184 |     0, 0, 0, 0, 0, 0, 0, 0, 20, 0, 0, 24, 0, 0, 20, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 20, 2, 24, 2, 20,
185 |     0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2, 53, 56, 53, 2, 0, 0, 0, 0, 0, 0, 24, 24, 24, 24, 24, 24, 56,
186 |     0, 56, 24, 24, 24, 24, 24, 24, 0, 0, 0, 0, 0, 0, 2, 53, 56, 53, 2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
187 |     0, 20, 2, 24, 2, 20, 0, 0, 0, 0, 0, 0, 0, 0, 20, 0, 0, 24, 0, 0, 20, 0, 0, 0, 0, 0, 0, 0, 0,
188 |     0, 20, 0, 0, 0, 24, 0, 0, 0, 20, 0, 0, 0, 0, 20, 0, 0, 0, 0, 24, 0, 0, 0, 0, 20, 0, 0, 0,
189 |     0, 20, 0, 0, 0, 0, 0, 24, 0, 0, 0, 0, 20, 0, 0, 20, 0, 0, 0, 0, 24, 0, 0, 0, 0, 0, 0,
190 |     20,
191 | ],
192 | RAYS = [
193 |     17, 0, 0, 0, 0, 0, 0, 0, 16, 0, 0, 0, 0, 0, 0, 15, 0, 0, 17, 0, 0, 0, 0, 0, 16, 0, 0, 0, 0, 0, 15,
194 |     0, 0, 0, 0, 17, 0, 0, 0, 0, 16, 0, 0, 0, 0, 15, 0, 0, 0, 0, 17, 0, 0, 0, 16, 0, 0, 0, 15,
195 |     0, 0, 0, 0, 0, 0, 0, 0, 17, 0, 0, 16, 0, 0, 15, 0, 0, 0, 0, 0, 0, 17, 0, 16, 0, 15,
196 |     0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 17, 16, 15, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 0, -1,
197 |     -1, -1, -1, -1, -1, -1, 0, 0, 0, 0, 0, 0, -15, -16, -17, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
198 |     -15, 0, -16, 0, -17, 0, 0, 0, 0, 0, 0, 0, 0, -15, 0, 0, -16, 0, 0, -17, 0, 0, 0, 0, 0, 0,
199 |     0, 0, -15, 0, 0, 0, -16, 0, 0, 0, -17, 0, 0, 0, 0, 0, -15, 0, 0, 0, 0, -16, 0, 0, 0, -17,
200 |     0, 0, 0, 0, -15, 0, 0, 0, 0, 0, -16, 0, 0, 0, 0, -17, 0, 0, -15, 0, 0, 0, 0, -16, 0, 0,
201 |     0, 0, 0, 0, -17,
202 | ],
203 | PIECE_MASKS = { p: 1, n: 2, b: 4, r: 8, q: 16, k: 32 },
204 | PROMOTIONS = ['n', 'b', 'r', 'q'],
205 | ROOKS = {
206 |     w: [
207 |         { square: 0x88.a1, flag: BITS.QSIDE_CASTLE },
208 |         { square: 0x88.h1, flag: BITS.KSIDE_CASTLE },
209 |     ],
210 |     b: [
211 |         { square: 0x88.a8, flag: BITS.QSIDE_CASTLE },
212 |         { square: 0x88.h8, flag: BITS.KSIDE_CASTLE },
213 |     ],
214 | },
215 | SECOND_RANK = { b: 1, w: 6 };
216 | function rank(square) {
217 |     return square >> 4;
218 | }
219 | function file(square) {

```

```

220 |   ·return·15·&·square;
221 | }
222 | function·isDigit(c)·{
223 |   ·return·-1·===·'0123456789'.indexOf(c);
224 | }
225 | function·algebraic(square)·{
226 |   ·const·f·=·file(square),
227 |   ··r·=·rank(square);
228 |   ·return·'abcdefgh'.substring(f,·f·+·1)·+·'87654321'.substring(r,·r·+·1);
229 | }
230 | function·swapColor(color)·{
231 |   ·return·'w'·===·color·?·'b'·:·'w';
232 | }
233 | function·addMove(moves,·color,·from,·to,·piece,·captured·=·void·0,·flags·=·BITS.NORMAL)·{
234 |   ·const·r·=·rank(to);
235 |   ·if·('p'·===·piece·||·(7·===·r·&·0·===·r))
236 |   ··moves.push({
237 |   ···color:·color,
238 |   ···from:·from,
239 |   ···to:·to,
240 |   ···piece:·piece,
241 |   ···captured:·captured,
242 |   ···flags:·flags,
243 |   ··});
244 |   ·else
245 |   ··for·(let·i·=·0;·i·<·PROMOTIONS.length;·i++)·{
246 |   ···const·promotion·=·PROMOTIONS[i];
247 |   ···moves.push({
248 |   ····color:·color,
249 |   ····from:·from,
250 |   ····to:·to,
251 |   ····piece:·piece,
252 |   ····captured:·captured,
253 |   ····promotion:·promotion,
254 |   ····flags:·flags·|·BITS.PROMOTION,
255 |   ···});
256 |   ··}
257 | }
258 | class·Chess·{
259 |   ·_board·=·new·Array(128);
260 |   ·_turn·=·'w';
261 |   ·_kings·=·{·w:·-1,·b:·-1};
262 |   ·_epSquare·=·-1;
263 |   ·_halfMoves·=·0;
264 |   ·_moveNumber·=·0;
265 |   ·_history·=·[];
266 |   ·_castling·=·{·w:·0,·b:·0};
267 |   ·_hash·=·0n;
268 |   ·_positionCount·=·new·Map();
269 |   ·constructor(fen·=·DEFAULT_POSITION,·{·skipValidation:·skipValidation·=·!1·}·=·{})·{
270 |   ··this.load(fen,·{·skipValidation:·skipValidation});
271 |   ·}
272 |   ·clear()·{
273 |   ··((this._board·=·new·Array(128)),
274 |   ···(this._kings·=·{·w:·-1,·b:·-1}),
275 |   ···(this._turn·=·'w'),
276 |   ···(this._castling·=·{·w:·0,·b:·0}),
277 |   ···(this._epSquare·=·-1),
278 |   ···(this._halfMoves·=·0),
279 |   ···(this._moveNumber·=·1),
280 |   ···(this._history·=·[]),
281 |   ···(this._hash·=·this._computeHash()),
282 |   ···(this._positionCount·=·new·Map()));
283 |   ·}
284 |   ·load(fen,·{·skipValidation:·skipValidation·=·!1·}·=·{})·{
285 |   ··let·tokens·=·fen.split(/\s+/);
286 |   ··if·(tokens.length·≥·2·&&·tokens.length·<·6)·{
287 |   ···const·adjustments·=·['-',·'-',·'0',·'1'];
288 |   ···fen·=·tokens.concat(adjustments.slice(-(6·-·tokens.length))).join('·');
289 |   ··}
290 |   ··if·((((tokens·=·fen.split(/\s+/)),·!skipValidation))·{

```

```

291 |     const { ok: ok, error: error } = (function validateFen(fen) {
292 |         const tokens = fen.split(/\s+/);
293 |         if (6 !== tokens.length)
294 |             return { ok: !1, error: 'Invalid FEN: must contain six space-delimited fields' };
295 |         const moveNumber = parseInt(tokens[5], 10);
296 |         if (isNaN(moveNumber) || moveNumber < 0)
297 |             return { ok: !1, error: 'Invalid FEN: move number must be a positive integer' };
298 |         const halfMoves = parseInt(tokens[4], 10);
299 |         if (isNaN(halfMoves) || halfMoves < 0)
300 |             return {
301 |                 ok: !1,
302 |                 error: 'Invalid FEN: half move counter number must be a non-negative integer',
303 |             };
304 |         if (!/^(?![abcdefgh][36])$/.test(tokens[3]))
305 |             return { ok: !1, error: 'Invalid FEN: en-passant square is invalid' };
306 |         if (/^[kKqQ-]/.test(tokens[2]))
307 |             return { ok: !1, error: 'Invalid FEN: castling availability is invalid' };
308 |         if (!/^(w|b)$/.test(tokens[1]))
309 |             return { ok: !1, error: 'Invalid FEN: side-to-move is invalid' };
310 |         const rows = tokens[0].split('/');
311 |         if (8 !== rows.length)
312 |             return { ok: !1, error: 'Invalid FEN: piece data does not contain 8 '/'-delimited rows' };
313 |         for (let i = 0; i < rows.length; i++) {
314 |             let sumFields = 0,
315 |                 previousWasNumber = !1;
316 |             for (let k = 0; k < rows[i].length; k++)
317 |                 if (isDigit(rows[i][k])) {
318 |                     if (previousWasNumber)
319 |                         return { ok: !1, error: 'Invalid FEN: piece data is invalid (consecutive number)' };
320 |                     (sumFields += parseInt(rows[i][k], 10)), (previousWasNumber = !0);
321 |                 } else {
322 |                     if (!/^[prnbqkPRNBQK]$/.test(rows[i][k]))
323 |                         return { ok: !1, error: 'Invalid FEN: piece data is invalid (invalid piece)' };
324 |                     (sumFields += 1), (previousWasNumber = !1);
325 |                 }
326 |             if (8 !== sumFields)
327 |                 return {
328 |                     ok: !1,
329 |                     error: 'Invalid FEN: piece data is invalid (too many squares in rank)',
330 |                 };
331 |         }
332 |         if (('3' === tokens[3][1] && 'w' === tokens[1]) || ('6' === tokens[3][1] && 'b' === tokens[1]))
333 |             return { ok: !1, error: 'Invalid FEN: illegal en-passant square' };
334 |         const kings = [
335 |             { color: 'white', regex: /K/g },
336 |             { color: 'black', regex: /k/g },
337 |         ];
338 |         for (const { color: color, regex: regex } of kings) {
339 |             if (!regex.test(tokens[0]))
340 |                 return { ok: !1, error: 'Invalid FEN: missing ${color} king`' };
341 |             if ((tokens[0].match(regex) || []).length > 1)
342 |                 return { ok: !1, error: 'Invalid FEN: too many ${color} kings`' };
343 |         }
344 |         return Array.from(rows[0] + rows[7]).some((char) => 'P' === char.toUpperCase())
345 |             ? { ok: !1, error: 'Invalid FEN: some pawns are on the edge rows' }
346 |             : { ok: !0 };
347 |     })(fen);
348 |     if (!ok) throw new Error(error);
349 | }
350 | const position = tokens[0];
351 | let square = 0;
352 | this.clear();
353 | for (let i = 0; i < position.length; i++) {
354 |     const piece = position.charAt(i);
355 |     if ('/' === piece) square += 8;
356 |     else if (isDigit(piece)) square += parseInt(piece, 10);
357 |     else {
358 |         const color = piece < 'a' ? 'w' : 'b';
359 |         (this._put({ type: piece.toLowerCase(), color: color }, algebraic(square)), square++);
360 |     }
361 | }

```

```

362 | ((this._turn = tokens[1]),
363 |   tokens[2].indexOf('K') > -1 && (this._castling.w |= BITS.KSIDE_CASTLE),
364 |   tokens[2].indexOf('Q') > -1 && (this._castling.w |= BITS.QSIDE_CASTLE),
365 |   tokens[2].indexOf('k') > -1 && (this._castling.b |= BITS.KSIDE_CASTLE),
366 |   tokens[2].indexOf('q') > -1 && (this._castling.b |= BITS.QSIDE_CASTLE),
367 |   (this._epSquare = '-' === tokens[3] ? -1 : 0x88[tokens[3]]),
368 |   (this._halfMoves = parseInt(tokens[4], 10)),
369 |   (this._moveNumber = parseInt(tokens[5], 10)),
370 |   (this._hash = this._computeHash()),
371 |   this._incPositionCount());
372 | }
373 | fen({ forceEnpassantSquare: forceEnpassantSquare !== 1 } = {}) {
374 |   let empty = 0,
375 |       fen = '';
376 |   for (let i = 0x88.a8; i ≤ 0x88.h1; i++) {
377 |     if (this._board[i]) {
378 |       empty > 0 && ((fen += empty), (empty = 0));
379 |       const { color: color, type: piece } = this._board[i];
380 |       fen += 'w' === color ? piece.toUpperCase() : piece.toLowerCase();
381 |     } else empty++;
382 |     (i + 1) & 136 &&
383 |     (empty > 0 && (fen += empty), i !== 0x88.h1 && (fen += '/'), (empty = 0), (i += 8));
384 |   }
385 |   let castling = '';
386 |   (this._castling.w & BITS.KSIDE_CASTLE && (castling += 'K'),
387 |   this._castling.w & BITS.QSIDE_CASTLE && (castling += 'Q'),
388 |   this._castling.b & BITS.KSIDE_CASTLE && (castling += 'k'),
389 |   this._castling.b & BITS.QSIDE_CASTLE && (castling += 'q'),
390 |   (castling = castling || '-'));
391 |   let epSquare = '-';
392 |   if (-1 !== this._epSquare)
393 |     if (forceEnpassantSquare) epSquare = algebraic(this._epSquare);
394 |     else {
395 |       const bigPawnSquare = this._epSquare + ('w' === this._turn ? 16 : -16),
396 |           squares = [bigPawnSquare + 1, bigPawnSquare - 1];
397 |       for (const square of squares) {
398 |         if (136 & square) continue;
399 |         const color = this._turn;
400 |         if (this._board[square]?.color === color && 'p' === this._board[square]?.type) {
401 |           this._makeMove({
402 |             color: color,
403 |             from: square,
404 |             to: this._epSquare,
405 |             piece: 'p',
406 |             captured: 'p',
407 |             flags: BITS.EP_CAPTURE,
408 |           });
409 |           const isLegal = !this._isKingAttacked(color);
410 |           if ((this._undoMove(), isLegal)) {
411 |             epSquare = algebraic(this._epSquare);
412 |             break;
413 |           }
414 |         }
415 |       }
416 |     }
417 |   return [fen, this._turn, castling, epSquare, this._halfMoves, this._moveNumber].join(' ');
418 | }
419 | _pieceKey(i) {
420 |   if (!this._board[i]) return 0n;
421 |   const { color: color, type: type } = this._board[i],
422 |       typeIndex = { p: 0, n: 1, b: 2, r: 3, q: 4, k: 5 }[type];
423 |   return PIECE_KEYS[{'w': 0, 'b': 1}[color]][typeIndex][i];
424 | }
425 | _epKey() {
426 |   return -1 === this._epSquare ? 0n : EP_KEYS[7 & this._epSquare];
427 | }
428 | _castlingKey() {
429 |   const index = (this._castling.w >> 5) | (this._castling.b >> 3);
430 |   return CASTLING_KEYS[index];
431 | }
432 | _computeHash() {

```

```

433 |     let hash = 0n;
434 |     for (let i = 0x88.a8; i <= 0x88.h1; i++)
435 |     {
436 |         136 & i ? (i += 7) : this._board[i] && (hash ^= this._pieceKey(i));
437 |         return (
438 |             (hash ^= this._epKey()),
439 |             (hash ^= this._castlingKey()),
440 |             'b' === this._turn && (hash ^= SIDE_KEY),
441 |             hash
442 |         );
443 |     }
444 |     reset() {
445 |         this.load(DEFAULT_POSITION);
446 |     }
447 |     get(square) {
448 |         return this._board[0x88[square]];
449 |     }
450 |     _set(sq, piece) {
451 |         ((this._hash ^= this._pieceKey(sq)),
452 |         (this._board[sq] = piece),
453 |         (this._hash ^= this._pieceKey(sq)));
454 |     }
455 |     _put({ type: type, color: color }, square) {
456 |         if (-1 === 'pnbrqkPNBRQK'.indexOf(type.toLowerCase())) return !1;
457 |         if (!(square in 0x88)) return !1;
458 |         const sq = 0x88[square];
459 |         if ('k' === type && -1 !== this._kings[color] && this._kings[color] !== sq) return !1;
460 |         const currentPieceOnSquare = this._board[sq];
461 |         return (
462 |             currentPieceOnSquare &&
463 |             'k' === currentPieceOnSquare.type &&
464 |             (this._kings[currentPieceOnSquare.color] !== -1),
465 |             this._set(sq, { type: type, color: color }),
466 |             'k' === type && (this._kings[color] = sq),
467 |             !0
468 |         );
469 |     }
470 |     _clear(sq) {
471 |         ((this._hash ^= this._pieceKey(sq)), delete this._board[sq]);
472 |     }
473 |     _attacked(color, square, verbose) {
474 |         const attackers = [];
475 |         for (let i = 0x88.a8; i <= 0x88.h1; i++) {
476 |             if (136 & i) {
477 |                 i += 7;
478 |                 continue;
479 |             }
480 |             if (void 0 === this._board[i] || this._board[i].color !== color) continue;
481 |             const piece = this._board[i],
482 |                 difference = i - square;
483 |             if (0 === difference) continue;
484 |             const index = difference + 119;
485 |             if (ATTACKS[index] & PIECE_MASKS[piece.type]) {
486 |                 if ('p' === piece.type) {
487 |                     if ((difference > 0 && 'w' === piece.color) || (difference <= 0 && 'b' === piece.color)) {
488 |                         if (!verbose) return !0;
489 |                         attackers.push(algebraic(i));
490 |                     }
491 |                     continue;
492 |                 }
493 |                 if ('n' === piece.type || 'k' === piece.type) {
494 |                     if (verbose) {
495 |                         attackers.push(algebraic(i));
496 |                         continue;
497 |                     }
498 |                     return !0;
499 |                 }
500 |                 const offset = RAYS[index];
501 |                 let j = i + offset,
502 |                     blocked = !1;
503 |                 for (; j !== square; ) {
504 |                     if (null !== this._board[j]) {

```



```

504 |         blocked = !0;
505 |         break;
506 |     }
507 |     j += offset;
508 | }
509 |     if (!blocked) {
510 |         if (verbose) {
511 |             attackers.push(algebraic(i));
512 |             continue;
513 |         }
514 |         return !0;
515 |     }
516 | }
517 | }
518 |     return !!verbose && attackers;
519 | }
520 |     attackers(square, attackedBy) {
521 |         return attackedBy
522 |             ? this._attacked(attackedBy, 0x88[square], !0)
523 |             : this._attacked(this._turn, 0x88[square], !0);
524 |     }
525 |     _isKingAttacked(color) {
526 |         const square = this._kings[color];
527 |         return -1 !== square && this._attacked(swapColor(color), square);
528 |     }
529 |     hash() {
530 |         return this._hash.toString(16);
531 |     }
532 |     isAttacked(square, attackedBy) {
533 |         return this._attacked(attackedBy, 0x88[square]);
534 |     }
535 |     isCheck() {
536 |         return this._isKingAttacked(this._turn);
537 |     }
538 |     inCheck() {
539 |         return this.isCheck();
540 |     }
541 |     isCheckmate() {
542 |         return this.isCheck() && 0 === this._moves().length;
543 |     }
544 |     isStalemate() {
545 |         return !this.isCheck() && 0 === this._moves().length;
546 |     }
547 |     isInsufficientMaterial() {
548 |         const pieces = {b: 0, n: 0, r: 0, q: 0, k: 0, p: 0},
549 |             bishops = [];
550 |         let numPieces = 0,
551 |             squareColor = 0;
552 |         for (let i = 0x88.a8; i ≤ 0x88.h1; i++) {
553 |             if (((squareColor = (squareColor + 1) % 2), 136 & i)) {
554 |                 i += 7;
555 |                 continue;
556 |             }
557 |             const piece = this._board[i];
558 |             piece &&
559 |                 ((pieces[piece.type] = piece.type in pieces ? pieces[piece.type] + 1 : 1),
560 |                 'b' === piece.type && bishops.push(squareColor),
561 |                 numPieces++);
562 |         }
563 |         if (2 === numPieces) return !0;
564 |         if (3 === numPieces && (1 === pieces.b || 1 === pieces.n)) return !0;
565 |         if (numPieces === pieces.b + 2) {
566 |             let sum = 0;
567 |             const len = bishops.length;
568 |             for (let i = 0; i < len; i++) sum += bishops[i];
569 |             if (0 === sum || sum === len) return !0;
570 |         }
571 |         return !1;
572 |     }
573 |     isThreefoldRepetition() {
574 |         return this._getPositionCount(this._hash) ≥ 3;

```

```

575 |     }
576 |     isDrawByFiftyMoves() {
577 |         return this._halfMoves ≥ 100;
578 |     }
579 |     isDraw() {
580 |         return (
581 |             this.isDrawByFiftyMoves() ||
582 |             this.isStalemate() ||
583 |             this.isInsufficientMaterial() ||
584 |             this.isThreefoldRepetition()
585 |         );
586 |     }
587 |     isGameOver() {
588 |         return this.isCheckmate() || this.isDraw();
589 |     }
590 |     moves({ verbose: verbose = !1, square: square, piece: piece } = {}) {
591 |         const moves = this._moves({ square: square, piece: piece });
592 |         return verbose
593 |             ? moves.map((move) => new Move(this, move))
594 |             : moves.map((move) => this._moveToSan(move, moves));
595 |     }
596 |     _moves({ legal: legal = !0, piece: piece, square: square } = {}) {
597 |         const forSquare = square ? square.toLowerCase() : void 0,
598 |             forPiece = piece ? piece.toLowerCase(),
599 |             moves = [],
600 |             us = this._turn,
601 |             them = swapColor(us);
602 |         let firstSquare = 0x88.a8,
603 |             lastSquare = 0x88.h1,
604 |             singleSquare = !1;
605 |         if (forSquare) {
606 |             if (!(forSquare in 0x88)) return [];
607 |             ((firstSquare = lastSquare = 0x88[forSquare]), (singleSquare = !0));
608 |         }
609 |         for (let from = firstSquare; from ≤ lastSquare; from++) {
610 |             if (136 & from) {
611 |                 from += 7;
612 |                 continue;
613 |             }
614 |             if (!this._board[from] || this._board[from].color === them) continue;
615 |             const { type: type } = this._board[from];
616 |             let to;
617 |             if ('p' === type) {
618 |                 if (forPiece && forPiece !== type) continue;
619 |                 ((to = from + PAWN_OFFSETS[us][0]),
620 |                 this._board[to] ||
621 |                 (addMove(moves, us, from, to, 'p'),
622 |                 (to = from + PAWN_OFFSETS[us][1]),
623 |                 SECOND_RANK[us] !== rank(from) ||
624 |                 this._board[to] ||
625 |                 addMove(moves, us, from, to, 'p', void 0, BITS.BIG_PAWN)));
626 |                 for (let j = 2; j < 4; j++)
627 |                     ((to = from + PAWN_OFFSETS[us][j]),
628 |                     136 & to ||
629 |                     (this._board[to] ? color === them
630 |                     ? addMove(moves, us, from, to, 'p', this._board[to].type, BITS.CAPTURE)
631 |                     : to === this._epSquare &&
632 |                     addMove(moves, us, from, to, 'p', 'p', BITS.EP_CAPTURE))));
633 |             } else {
634 |                 if (forPiece && forPiece !== type) continue;
635 |                 for (let j = 0, len = PIECE_OFFSETS[type].length; j < len; j++) {
636 |                     const offset = PIECE_OFFSETS[type][j];
637 |                     for (to = from; (to += offset), !(136 & to);) {
638 |                         if (this._board[to]) {
639 |                             if (this._board[to].color === us) break;
640 |                             addMove(moves, us, from, to, type, this._board[to].type, BITS.CAPTURE);
641 |                             break;
642 |                         }
643 |                         if ((addMove(moves, us, from, to, type), 'n' === type || 'k' === type)) break;
644 |                     }
645 |                 }

```

```

646 |     }
647 | }
648 | if (
649 |     !(
650 |         (void 0 !== forPiece & 'k' !== forPiece) ||
651 |         (singleSquare & lastSquare !== this._kings[us])
652 |     )
653 | ) {
654 |     if (this._castling[us] & BITS.KSIDE_CASTLE) {
655 |         const castlingFrom = this._kings[us],
656 |             castlingTo = castlingFrom + 2;
657 |         this._board[castlingFrom + 1] ||
658 |             this._board[castlingTo] ||
659 |             this._attacked(them, this._kings[us]) ||
660 |             this._attacked(them, castlingFrom + 1) ||
661 |             this._attacked(them, castlingTo) ||
662 |             addMove(moves, us, this._kings[us], castlingTo, 'k', void 0, BITS.KSIDE_CASTLE);
663 |     }
664 |     if (this._castling[us] & BITS.QSIDE_CASTLE) {
665 |         const castlingFrom = this._kings[us],
666 |             castlingTo = castlingFrom - 2;
667 |         this._board[castlingFrom - 1] ||
668 |             this._board[castlingFrom - 2] ||
669 |             this._board[castlingFrom - 3] ||
670 |             this._attacked(them, this._kings[us]) ||
671 |             this._attacked(them, castlingFrom - 1) ||
672 |             this._attacked(them, castlingTo) ||
673 |             addMove(moves, us, this._kings[us], castlingTo, 'k', void 0, BITS.QSIDE_CASTLE);
674 |     }
675 | }
676 | if (!legal || -1 === this._kings[us]) return moves;
677 | const legalMoves = [];
678 | for (let i = 0, len = moves.length; i < len; i++)
679 |     (this._makeMove(moves[i]),
680 |         this._isKingAttacked(us) || legalMoves.push(moves[i]),
681 |         this._undoMove());
682 | return legalMoves;
683 | }
684 | move(move) {
685 |     let moveObj = null;
686 |     const moves = this._moves();
687 |     for (let i = 0, len = moves.length; i < len; i++)
688 |         if (
689 |             move.from === algebraic(moves[i].from) &
690 |             move.to === algebraic(moves[i].to) &
691 |             (!('promotion' in moves[i]) || move.promotion === moves[i].promotion)
692 |         ) {
693 |             moveObj = moves[i];
694 |             break;
695 |         }
696 |     if (!moveObj) throw new Error(`Invalid move: ${JSON.stringify(move)}`);
697 |     const prettyMove = new Move(this, moveObj);
698 |     return (this._makeMove(moveObj), this._incPositionCount(), prettyMove);
699 | }
700 | _push(move) {
701 |     this._history.push({
702 |         move: move,
703 |         kings: { b: this._kings.b, w: this._kings.w },
704 |         turn: this._turn,
705 |         castling: { b: this._castling.b, w: this._castling.w },
706 |         epSquare: this._epSquare,
707 |         halfMoves: this._halfMoves,
708 |         moveNumber: this._moveNumber,
709 |     });
710 | }
711 | _movePiece(from, to) {
712 |     ((this._hash ^= this._pieceKey(from)),
713 |         (this._board[to] = this._board[from]),
714 |         delete this._board[from],
715 |         (this._hash ^= this._pieceKey(to)));
716 | }

```

```

717 |     _makeMove(move)·{
718 |         const us = this._turn,
719 |         them = swapColor(us);
720 |         if·((this._push(move), move.flags & BITS.NULL_MOVE))
721 |             return·(
722 |                 'b' === us && this._moveNumber++,
723 |                 this._halfMoves++,
724 |                 (this._turn = them),
725 |                 void·(this._epSquare = -1)
726 |             );
727 |         if·(
728 |             ((this._hash ^= this._epKey()),
729 |             (this._hash ^= this._castlingKey()),
730 |             move.captured && (this._hash ^= this._pieceKey(move.to)),
731 |             this._movePiece(move.from, move.to),
732 |             move.flags & BITS.EP_CAPTURE &&
733 |             ('b' === this._turn ? this._clear(move.to - 16) : this._clear(move.to + 16)),
734 |             move.promotion &&
735 |             (this._clear(move.to), this._set(move.to, { type: move.promotion, color: us })),
736 |             'k' === this._board[move.to].type)
737 |         )·{
738 |             if·(((this._kings[us] = move.to), move.flags & BITS.KSIDE_CASTLE))·{
739 |                 const castlingTo = move.to - 1,
740 |                 castlingFrom = move.to + 1;
741 |                 this._movePiece(castlingFrom, castlingTo);
742 |             } else if·(move.flags & BITS.QSIDE_CASTLE)·{
743 |                 const castlingTo = move.to + 1,
744 |                 castlingFrom = move.to - 2;
745 |                 this._movePiece(castlingFrom, castlingTo);
746 |             }
747 |             this._castling[us] = 0;
748 |         }
749 |         if·(this._castling[us])
750 |             for·(let i = 0, len = ROOKS[us].length; i < len; i++)
751 |                 if·(move.from === ROOKS[us][i].square && this._castling[us] & ROOKS[us][i].flag)·{
752 |                     this._castling[us] ^= ROOKS[us][i].flag;
753 |                     break;
754 |                 }
755 |         if·(this._castling[them])
756 |             for·(let i = 0, len = ROOKS[them].length; i < len; i++)
757 |                 if·(move.to === ROOKS[them][i].square && this._castling[them] & ROOKS[them][i].flag)·{
758 |                     this._castling[them] ^= ROOKS[them][i].flag;
759 |                     break;
760 |                 }
761 |         if·(((this._hash ^= this._castlingKey()), move.flags & BITS.BIG_PAWN))·{
762 |             let epSquare;
763 |             ((epSquare = 'b' === us ? move.to - 16 : move.to + 16),
764 |             ((move.to - 1) & 136 ||
765 |             'p' !== this._board[move.to - 1] ? type ||
766 |             this._board[move.to - 1] ? color !== them) &&
767 |             ((move.to + 1) & 136 ||
768 |             'p' !== this._board[move.to + 1] ? type ||
769 |             this._board[move.to + 1] ? color !== them)
770 |             ? (this._epSquare = -1)
771 |             : ((this._epSquare = epSquare), (this._hash ^= this._epKey())));
772 |         } else this._epSquare = -1;
773 |         ('p' === move.piece || move.flags & (BITS.CAPTURE | BITS.EP_CAPTURE)
774 |         ? (this._halfMoves = 0)
775 |         : this._halfMoves++,
776 |         'b' === us && this._moveNumber++,
777 |         (this._turn = them),
778 |         (this._hash ^= SIDE_KEY));
779 |     }
780 |     undo()·{
781 |         const hash = this._hash,
782 |         move = this._undoMove();
783 |         if·(move)·{
784 |             const prettyMove = new Move(this, move);
785 |             return·(this._decPositionCount(hash), prettyMove);
786 |         }
787 |         return·null;

```

```

788 |     }
789 |     _undoMove() {
790 |         const old = this._history.pop();
791 |         if (void 0 === old) return null;
792 |         ((this._hash ^= this._epKey()), (this._hash ^= this._castlingKey()));
793 |         const move = old.move;
794 |         ((this._kings = old.kings),
795 |         (this._turn = old.turn),
796 |         (this._castling = old.castling),
797 |         (this._epSquare = old.epSquare),
798 |         (this._halfMoves = old.halfMoves),
799 |         (this._moveNumber = old.moveNumber),
800 |         (this._hash ^= this._epKey()),
801 |         (this._hash ^= this._castlingKey()),
802 |         (this._hash ^= SIDE_KEY));
803 |         const us = this._turn,
804 |               them = swapColor(us);
805 |         if (move.flags & BITS.NULL_MOVE) return move;
806 |         if (
807 |             (this._movePiece(move.to, move.from),
808 |             move.piece && (this._clear(move.from), this._set(move.from, { type: move.piece, color: us })),
809 |             move.captured)
810 |         )
811 |             if (move.flags & BITS.EP_CAPTURE) {
812 |                 let index;
813 |                 ((index = 'b' === us ? move.to - 16 : move.to + 16),
814 |                 this._set(index, { type: 'p', color: them }));
815 |             } else this._set(move.to, { type: move.captured, color: them });
816 |         if (move.flags & (BITS.KSIDE_CASTLE | BITS.QSIDE_CASTLE)) {
817 |             let castlingTo, castlingFrom;
818 |             (move.flags & BITS.KSIDE_CASTLE
819 |             ? ((castlingTo = move.to + 1), (castlingFrom = move.to - 1))
820 |             : ((castlingTo = move.to - 2), (castlingFrom = move.to + 1))),
821 |             this._movePiece(castlingFrom, castlingTo));
822 |         }
823 |         return move;
824 |     }
825 |     _moveToSan(move, moves) {
826 |         let output = '';
827 |         if (move.flags & BITS.KSIDE_CASTLE) output = 'O-O';
828 |         else if (move.flags & BITS.QSIDE_CASTLE) output = 'O-O-O';
829 |         else {
830 |             if ('p' !== move.piece) {
831 |                 const disambiguator = (function getDisambiguator(move, moves) {
832 |                     const from = move.from,
833 |                           to = move.to,
834 |                           piece = move.piece;
835 |                     let ambiguities = 0,
836 |                         sameRank = 0,
837 |                         sameFile = 0;
838 |                     for (let i = 0, len = moves.length; i < len; i++) {
839 |                         const ambigFrom = moves[i].from,
840 |                               ambigTo = moves[i].to;
841 |                         piece === moves[i].piece &&
842 |                         from !== ambigFrom &&
843 |                         to === ambigTo &&
844 |                         (ambiguities++,
845 |                         rank(from) === rank(ambigFrom) && sameRank++,
846 |                         file(from) === file(ambigFrom) && sameFile++);
847 |                     }
848 |                     return ambiguities > 0
849 |                         ? sameRank > 0 && sameFile > 0
850 |                         ? algebraic(from)
851 |                         : sameFile > 0
852 |                         ? algebraic(from).charAt(1)
853 |                         : algebraic(from).charAt(0)
854 |                         : '';
855 |                 })(move, moves);
856 |                 output += move.piece.toUpperCase() + disambiguator;
857 |             }
858 |             (move.flags & (BITS.CAPTURE | BITS.EP_CAPTURE) &&

```

```

859 |         ('p' === move.piece && (output += algebraic(move.from)[0]), (output += 'x')),
860 |         (output += algebraic(move.to)),
861 |         move.promotion && (output += '=' + move.promotion.toUpperCase()));
862 |     }
863 |     return (
864 |         this._makeMove(move),
865 |         this.isCheck() && (this.isCheckmate() ? (output += '#') : (output += '+')),
866 |         this._undoMove(),
867 |         output
868 |     );
869 | }
870 | turn() {
871 |     return this._turn;
872 | }
873 | board() {
874 |     const output = [];
875 |     let row = [];
876 |     for (let i = 0x88.a8; i ≤ 0x88.h1; i++)
877 |         (null === this._board[i]
878 |         ? row.push(null)
879 |         : row.push({
880 |             square: algebraic(i),
881 |             type: this._board[i].type,
882 |             color: this._board[i].color,
883 |         })),
884 |         (i + 1) & 136 && (output.push(row), (row = []), (i += 8));
885 |     return output;
886 | }
887 | squareColor(square) {
888 |     if (square in 0x88) {
889 |         const sq = 0x88[square];
890 |         return (rank(sq) + file(sq)) % 2 === 0 ? 'light' : 'dark';
891 |     }
892 |     return null;
893 | }
894 | history({ verbose: verbose = !1 } = {}) {
895 |     const reversedHistory = [],
896 |           moveHistory = [];
897 |     for (; this._history.length > 0; ) reversedHistory.push(this._undoMove());
898 |     for (;;) {
899 |         const move = reversedHistory.pop();
900 |         if (!move) break;
901 |         (verbose
902 |         ? moveHistory.push(new Move(this, move))
903 |         : moveHistory.push(this._moveToSan(move, this._moves()))),
904 |         this._makeMove(move);
905 |     }
906 |     return moveHistory;
907 | }
908 | _getPositionCount(hash) {
909 |     return this._positionCount.get(hash) ?? 0;
910 | }
911 | _incPositionCount() {
912 |     this._positionCount.set(this._hash, (this._positionCount.get(this._hash) ?? 0) + 1);
913 | }
914 | _decPositionCount(hash) {
915 |     const currentCount = this._positionCount.get(hash) ?? 0;
916 |     1 === currentCount
917 |     ? this._positionCount.delete(hash)
918 |     : this._positionCount.set(hash, currentCount - 1);
919 | }
920 | moveNumber() {
921 |     return this._moveNumber;
922 | }
923 | }
924 | const AI_PRESETS = {
925 |     easy: { preset: 'easy', maxDepth: 2, timeLimitMs: 750, randomness: 30, quiescence: !0 },
926 |     normal: { preset: 'normal', maxDepth: 5, timeLimitMs: 5e3, randomness: 30, quiescence: !0 },
927 |     hard: { preset: 'hard', maxDepth: 5, timeLimitMs: 5e3, randomness: 30, quiescence: !0 },
928 | },
929 | DEFAULT_AI_SETTINGS = { ... AI_PRESETS.normal };

```



```

    ↪ astling.availability.is.invalid");if(!/^(\w\b)$/.test(tokens[1]))return{ok:!1,error:"Invalid FEN: side-to-move is invalid"};const rows=tokens[0].split("/");if(8≠rows.length)return{ok:!1,error:"Invalid FEN: piece data does not contain 8 '\ '/' -delimited rows"};for(let i=0;i<rows.length;i++){let sumFields=0,previousWasNumber=!1;for(let k=0;k<rows[i].length;k++)if(isDigit(rows[i][k])){if(previousWasNumber)return{ok:!1,error:"Invalid FEN: piece data is invalid (consecutive number)"};sumFields+=parseInt(rows[i][k],10),previousWasNumber=!0}else{if(!/^[\prnbqkPRNBQK]$/.test(rows[i][k]))return{ok:!1,error:"Invalid FEN: piece data is invalid (invalid piece)"};sumFields+=1,previousWasNumber=!1;if(8≠sumFields)return{ok:!1,error:"Invalid FEN: piece data is invalid (too many squares in rank)"}if("3"=tokens[3][1]&&"w"=tokens[1][1]&&"6"=tokens[3][1]&&"b"=tokens[1])return{ok:!1,error:"Invalid FEN: illegal en-passant square"};const kings=[{color:"white",regex:/K/g},{color:"black",regex:/k/g}];for(const{color:color,regex:regex}of kings){if(!regex.test(tokens[0]))return{ok:!1,error:"Invalid FEN: missing ${color} king"};if((tokens[0].match(regex)||[]).length>1)return{ok:!1,error:"Invalid FEN: too many ${color} kings"}}return Array.from(rows[0]+rows[7]).some(char⇒"P"=char.toUpperCase())?{ok:!1,error:"Invalid FEN: some pawns are on the edge rows"}:{ok:!0}}(fen);if(!ok)throw new Error(error)}const position=tokens[0];let square=0;this.clear();for(let i=0;i<position.length;i++){const piece=position.charAt(i);if("/"=piece)square+=8;else if(isDigit(piece))square+=parseInt(piece,10);else{const color=piece<"a"? "w": "b";this._put({type:piece.toLowerCase(),color:color},algebraic(square)),square++}this._turn=tokens[1],tokens[2].indexOf("K")>1&&(this._castling.wl=BITS.KSIDE_CASTLE),tokens[2].indexOf("Q")>1&&(this._castling.wl=BITS.QSIDE_CASTLE),tokens[2].indexOf("k")>1&&(this._castling.bl=BITS.KSIDE_CASTLE),tokens[2].indexOf("q")>1&&(this._castling.bl=BITS.QSIDE_CASTLE),this._epSquare="-"=tokens[3]?-1:0x88[tokens[3]],this._halfMoves=parseInt(tokens[4],10),this._moveNumber=parseInt(tokens[5],10),this._hash=this._computeHash(),this._incPositionCount()}fen({forceEnpassantSquare:forceEnpassantSquare=!1}={}){let empty=0,fen="";for(let i=0x88.a8;i<=0x88.h1;i++){if(this._board[i]){empty>0&&(fen+=empty,empty=0);const{color:color,type:piece}=this._board[i];fen+= "w"=color?piece.toUpperCase():piece.toLowerCase()}else empty++;i+1&136&&(empty>0&&(fen+=empty),i≠0x88.h1&&(fen+= "/"),empty=0,i+=8)}let castling="";this._castling.w&BITS.KSIDE_CASTLE&&(castling+="K"),this._castling.w&BITS.QSIDE_CASTLE&&(castling+="Q"),this._castling.b&BITS.KSIDE_CASTLE&&(castling+="k"),this._castling.b&BITS.QSIDE_CASTLE&&(castling+="q"),castling=castling|| "-";let epSquare="-";if(-1≠this._epSquare)if(forceEnpassantSquare)epSquare=algebraic(this._epSquare);else{const bigPawnSquare=this._epSquare+"w"=this._turn?16:-16,squares=[bigPawnSquare+1,bigPawnSquare-1];for(const square of squares){if(136&square)continue;const color=this._turn;if(this._board[square]?.color=color&&"p"=this._board[square]?.type){this._makeMove({color:color,from:square,to:this._epSquare,piece:"p",captured:"p",flags:BITS.EP_CAPTURE});const isLegal=!this._isKingAttacked(color);if(this._undoMove(),isLegal){epSquare=algebraic(this._epSquare);break}}}}return[fen,this._turn,castling,epSquare,this._halfMoves,this._moveNumber].join(" ")_pieceKey(i){if(!this._board[i])return 0n;const{color:color,type:type}=this._board[i],typeIndex={p:0,n:1,b:2,r:3,q:4,k:5}[type];return PIECE_KEYS[{w:0,b:1}[color]][typeIndex][i]_epKey(){return-1=this._epSquare?0n:EP_KEYS[7&this._epSquare]}_castlingKey(){const index=this._castling.w>>5,this._castling.b>>3;return CASTLING_KEYS[index]}_computeHash(){let hash=0n;for(let i=0x88.a8;i<=0x88.h1;i++){136&i?i+=7:this._board[i]&&(hash^=this._pieceKey(i));return hash^=this._epKey(),hash^=this._castlingKey(),"b"=this._turn&&(hash^=SIDE_KEY),hash}reset(){this.load(DEFAULT_POSITION)}get(square){return this._board[0x88[square]]}_set(sq,piece){this._hash^=this._pieceKey(sq),this._board[sq]=piece,this._hash^=this._pieceKey(sq)}_put({type:type,color:color},square){if(-1=pbnrqkPNBRQK".indexOf(type.toLowerCase()))return!1;if(!(square in 0x88))return!1;const sq=0x88[square];if("k"=type-1≠this._kings[color]&&this._kings[color]≠sq)return!1;const currentPieceOnSquare=this._board[sq];return currentPieceOnSquare&&"k"=currentPieceOnSquare.type&&(this._kings[currentPieceOnSquare.color]=-1),this._set(sq,{type:type,color:color}),"k"=type&&(this._kings[color]=sq,!0)_clear(sq){this._hash^=this._pieceKey(sq),delete this._board[sq]}_attacked(color,square,verbose){const attackers=[];for(let i=0x88.a8;i<=0x88.h1;i++){if(136&i){i+=7;continue}if(void 0=this._board[i]|| this._board[i].color≠color)continue;const piece=this._board[i],difference=i-square;if(0=difference)continue;const index=difference+119;if(ATTACKS[index]&PIECE_MASKS[piece.type]){if("p"=piece.type){if(difference>0&&"w"=piece.color|| difference<=0&&"b"=piece.color){if(!verbose)return!0;attackers.push(algebraic(i))}continue}if("n"=piece.type|| "k"=piece.type){if(verbose){attackers.push(algebraic(i));continue}return!0}const offset=RAYS[index];let j=i+offset,blocked=!1;for(;j≠square;){if(null≠this._board[j]){blocked=!0;break}j+=offset}if(!blocked){if(verbose){attackers.push(algebraic(i));continue}return!0}}return!! verbose&&attackers.length>0}isKingAttacked(color){const square=this._kings[color];return-1≠square&&this._attacked(swapColor(color),square)}hash(){return this._hash.toString(16)}isAttacked(square,attackedBy){return this._attacked(attackedBy,0x88[square])}isCheck(){return this._isKingAttacked(this._turn)}inCheck(){return this.isCheck()}isCheckmate(){return this.isCheck()&&this._moves().length>0}isStalemate(){return!this.isCheck()&&0=this._moves().length}isInsufficientMaterial(){const pieces={b:0,n:0,r:0,q:0,k:0,p:0},bishops=[];let numPieces=0,squareColor=0;for(let i=0x88.a8;i<=0x88.h1;i++){if(squareColor=(squareColor+1)%2,136&i){i+=7;continue}const piece=this._board[i];piece&&(pieces[piece.type]=piece.type in pieces?pieces[piece.type]+1:1,"b"=piece.type&&bishops.push(squareColor),numPieces++)}if(2=numPieces)return!0;if(3=numPieces&&(1=pieces.b|| 1=pieces.n))return!0;if(numPieces=pieces.b+2){let sum=0;const len=bishops.length;for(let i=0;i<len;i++)sum+=bishops[i];if(0=sum|| sum=len)return!0}return!1}isThreefoldRepetition(){return this._getPositionCount(this._hash)>=3}isDrawByFiftyMoves(){return this._halfMoves>=100}isDraw(){return this.isDrawByFiftyMoves()|| this.isStalemate()|| this.isInsufficientMaterial()|| this.isThreefoldRepetition()}isGameOver(){return this.isCheckmate()|| this.isDraw()}moves({verbose:verbose=!1,square:square,piece:piece}={}){const moves=this._moves({square:square,piece:piece});return verbose?moves.map(move⇒new Move(this,move)):moves.map(move⇒this._moveToSan(move,moves))}_moves({legal:legal=!0,piece:piece,square:square}={}){const forSquare=square?square.toLowerCase():void 0,forPiece=piece?.toLowerCase(),moves=[],us=this._turn,them=swapColor(us);let firstSquare=0x88.a8,lastSquare=0x88.h1,singleSquare=!1;if(forSquare){if(!(forSquare in 0x88))return[];firstSquare=lastSquare=0x88[forSquare],singleSquare=!0}for(let from=firstSquare;from<=lastSquare;from++){if(136&from){from+=7;continue}if(!this._board[from]|| this._board[from].color≠them)continue;const{type:type}=this._board[from];let to;if("p"=type){if(forPiece&&forPiece≠type)continue;to=from+PAWN_OFFSETS[us][0],this._board[to]|| (addMove(moves,us,from,to,"p"),to=from+PAWN_OFFSETS[us][1],SECOND_RANK=us≠rank(from)|| this._board[to]|| addMove(moves,us,from,to,"p",void 0,BITS.BIG_PAWN));for(let t=t-j=2;j<4;j++)to=from+PAWN_OFFSETS[us][j],136&to|| (this._board[to]?.color=them?addMove(moves,us,from,to,"p",this._board[to].type,BITS.CAPTURE):to=this._epSquare&&addMove(moves,us,from,to,"p","p",BITS.EP_CAPTURE))}else if(forPiece&&forPiece≠type)continue;for(let j=0,len=PIECE_OFFSETS[type].length;j<len;j++){const offset=PIECE_OFFSETS[type][j];for(to=from;to+=offset,! (136&to);){if(this._board[to]){if(this._board[to].color=us)break;addMove(moves,us,from,to,type,this._board[to].type,BITS.CAPTURE);break}if(addMove(moves,us,from,to,type),"n"=type|| "k"=type)break}}if(!(void 0=forPiece&&"k"=forPiece|| singleSquare&&lastSquare≠this._kings[us])){if(this._castling[us]&BITS.KSIDE_CASTLE){const castlingFrom=thi

```



```

    s._kings[us],castlingTo=castlingFrom+2;this._board[castlingFrom+1]||this._board[castlingTo]||this._attacked(
    ngs[us])||this._attacked(them,castlingFrom+1)||this._attacked(them,castlingTo)||addMove(moves,us,this._kings[us],castling
    To,"k",void 0,BITS.KSIDE_CASTLE)}if(this._castling[us]&BITS.QSIDE_CASTLE){const castlingFrom=this._kings[us],castlingTo=c
    astlingFrom-2;this._board[castlingFrom-1]||this._board[castlingFrom-2]||this._board[castlingFrom-3]||this._attacked(them,
    this._kings[us])||this._attacked(them,castlingFrom-1)||this._attacked(them,castlingTo)||addMove(moves,us,this._kings[us],
    castlingTo,"k",void 0,BITS.QSIDE_CASTLE)}if(!legal||1===this._kings[us])return moves;const legalMoves=[];for(let i=0,le
    n=moves.length;i<len;i++)this._makeMove(moves[i]),this._isKingAttacked(us)||legalMoves.push(moves[i]),this._undoMove();re
    turn.legalMoves}move(move){let moveObj=null;const moves=this._moves();for(let i=0,len=moves.length;i<len;i++)if(move.from
    ===algebraic(moves[i].from)&&move.to===algebraic(moves[i].to)&&(!("promotion"in moves[i])||move.promotion===moves[i].prom
    otion)){moveObj=moves[i];break}if(!moveObj)throw new Error(`Invalid move: ${JSON.stringify(move)}`);const prettyMove=new
    Move(this,moveObj);return this._makeMove(moveObj),this._incPositionCount(),prettyMove.push(move){this._history.push({mov
    e:move,kings:{b:this._kings.b,w:this._kings.w},turn:this._turn,castling:{b:this._castling.b,w:this._castling.w},epSquare:
    this._epSquare,halfMoves:this._halfMoves,moveNumber:this._moveNumber}}}_movePiece(from,to){this._hash^=this._pieceKey(fro
    m),this._board[to]=this._board[from],delete this._board[from],this._hash^=this._pieceKey(to)}_makeMove(move){const us=thi
    s._turn,them=swapColor(us);if(this._push(move),move.flags&BITS.NULL_MOVE)return"b"===us&&this._moveNumber++,this._halfMov
    es++,this._turn=them,void(this._epSquare=-1);if(this._hash^=this._epKey(),this._hash^=this._castlingKey(),move.captured&
    (this._hash^=this._pieceKey(move.to)),this._movePiece(move.from,move.to),move.flags&BITS.EP_CAPTURE&&("b"===this._turn?th
    is._clear(move.to-16):this._clear(move.to+16)),move.promotion&&(this._clear(move.to),this._set(move.to,{type:move.promoti
    on,color:us})),this._board[move.to].type){if(this._kings[us]=move.to,move.flags&BITS.KSIDE_CASTLE){const castlingTo
    =move.to-1,castlingFrom=move.to+1;this._movePiece(castlingFrom,castlingTo)}else if(move.flags&BITS.QSIDE_CASTLE){const ca
    stlingTo=move.to+1,castlingFrom=move.to-2;this._movePiece(castlingFrom,castlingTo)}this._castling[us]=0;if(this._castling
    [us])for(let i=0,len=ROOKS[us].length;i<len;i++)if(move.from===ROOKS[us][i].square&&this._castling[us]&ROOKS[us][i].flag)
    {this._castling[us]^=ROOKS[us][i].flag;break}if(this._castling[them])for(let i=0,len=ROOKS[them].length;i<len;i++)if(move
    .to===ROOKS[them][i].square&&this._castling[them]&ROOKS[them][i].flag){this._castling[them]^=ROOKS[them][i].flag;break}if
    (this._hash^=this._castlingKey(),move.flags&BITS.BIG_PAWN){let epSquare,epSquare="b"===us?move.to-16:move.to+16,(move.to-
    1&136||"p"===this._board[move.to-1]??.type||this._board[move.to-1]??.color===them)&&(move.to+1&136||"p"===this._board[move
    .to+1]??.type||this._board[move.to+1]??.color===them)?this._epSquare=-1:(this._epSquare=epSquare,this._hash^=this._epKey())}
    else this._epSquare=-1;"p"===move.piece||move.flags&(BITS.CAPTURE|BITS.EP_CAPTURE)?this._halfMoves=0:this._halfMoves++,"b
    "===us&&this._moveNumber++,this._turn=them,this._hash^=SIDE_KEY}undo(){const hash=this._hash,move=this._undoMove();if(mov
    e){const prettyMove=new Move(this,move);return this._decPositionCount(hash),prettyMove)return null}_undoMove(){const old=
    this._history.pop();if(void 0===old)return null;this._hash^=this._epKey(),this._hash^=this._castlingKey();const move=old
    move;this._kings=old.kings,this._turn=old.turn,this._castling=old.castling,this._epSquare=old.epSquare,this._halfMoves=ol
    d.halfMoves,this._moveNumber=old.moveNumber,this._hash^=this._epKey(),this._hash^=this._castlingKey(),this._hash^=SIDE_KE
    Y;const us=this._turn,them=swapColor(us);if(move.flags&BITS.NULL_MOVE)return move;if(this._movePiece(move.to,move.from),m
    ove.piece&&(this._clear(move.from),this._set(move.from,{type:move.piece,color:us})),move.captured)if(move.flags&BITS.EP_C
    APTURE){let index,index="b"===us?move.to-16:move.to+16,this._set(index,{type:"p",color:them});}else this._set(move.to,{typ
    e:move.captured,color:them});if(move.flags&(BITS.KSIDE_CASTLE|BITS.QSIDE_CASTLE)){let castlingTo,castlingFrom;move.flags&
    BITS.KSIDE_CASTLE?(castlingTo=move.to+1,castlingFrom=move.to-1):(castlingTo=move.to-2,castlingFrom=move.to+1),this._moveP
    iece(castlingFrom,castlingTo)}return move}_moveToSan(move,moves){let output="";if(move.flags&BITS.KSIDE_CASTLE)output="O-
    0";else if(move.flags&BITS.QSIDE_CASTLE)output="O-0-O";else if("p"===move.piece){const disambiguator=function getDisambig
    uator(move,moves){const from=move.from,to=move.to,piece=move.piece;let ambiguities=0,sameRank=0,sameFile=0;for(let i=0,le
    n=moves.length;i<len;i++){const ambigFrom=moves[i].from,ambigTo=moves[i].to;piece===moves[i].piece&&from===ambigFrom&&to
    ===ambigTo&&(ambiguities++,rank(from)===rank(ambigFrom)&&sameRank++,file(from)===file(ambigFrom)&&sameFile++)}return ambig
    uities>0?sameRank>0&&sameFile>0?algebraic(from):sameFile>0?algebraic(from).charAt(1):algebraic(from).charAt(0):""}(move,m
    oves);output+=move.piece.toUpperCase()+disambiguator}move.flags&BITS.CAPTURE|BITS.EP_CAPTURE&&("p"===move.piece&&(output
    +=algebraic(move.from)[0]),output+="x"),output+=algebraic(move.to),move.promotion&&(output+=" "+move.promotion.toUpperCase
    se())}return this._makeMove(move),this._isCheck()&&(this._isCheckmate()?output+="#":output+="+"),this._undoMove(),output}tu
    rn(){return this._turn}board(){const output=[];let row=[];for(let i=0x88.a8;i<=0x88.h1;i++)null===this._board[i]?row.push(
    null):row.push({square:algebraic(i),type:this._board[i].type,color:this._board[i].color}),i+1&136&&(output.push(row),row=
    [],i+=8);return output}squareColor(square){if(square.in 0x88){const sq=0x88[square];return(rank(sq)+file(sq))%2===0?"light
    ":"dark"}return null}history({verbose:verbose=!1}=){const reversedHistory=[],moveHistory=[];for(;this._history.length>0
    ;){reversedHistory.push(this._undoMove());for(;;){const move=reversedHistory.pop();if(!move)break;verbose?moveHistory.push
    (new Move(this,move)):moveHistory.push(this._moveToSan(move,this._moves()))},this._makeMove(move)}return moveHistory}_getP
    ositionCount(hash){return this._positionCount.get(hash)??._incPositionCount(){this._positionCount.set(this._hash,this._
    positionCount.get(this._hash)??.+1)}_decPositionCount(hash){const currentCount=this._positionCount.get(hash)??.1;curr
    entCount?this._positionCount.delete(hash):this._positionCount.set(hash,currentCount-1)}moveNumber(){return this._moveNumb
    er}}const PIECE_VALUES={p:100,n:320,b:330,r:500,q:900,k:0};function searchRoot(chess,depth,ctx,previousBestMove){const ca
    ndidates=[],legalMoves=orderMoves(chess.moves({verbose:!0}),previousBestMove);for(const move of legalMoves){checkTime(ctx
    ),chess.move(toMoveCommand(move));const score=-negamax(chess,depth-1,-1e7,1e7,1,ctx);chess.undo(),candidates.push({move:t
    oMoveCommand(move),san:move.san,score:score})}if(0===candidates.length)throw new SearchTimeout;const principal=function s
    electBestRootMove(candidates){return candidates.reduce((currentBest,candidate)=>candidate.score>currentBest.score?candi
    date:currentBest)}(candidates),selection=function selectRootMove(candidates,settings,best){const level=settings.randomness/
    30>windowCp=20+150*level,floorScore=best.score-windowCp,ticketPower=4-1.5*level;if(function shouldForceBestMove(settings,
    best){return 0===settings.randomness||function isMateProtectedScore(score){return Math.abs(score)>=99e4}(best.score)||"ha
    rd"===settings.preset&&best.score<0}(settings,best)}return{selected:best,debug:createDeterministicDebug(candidates,best,b
    est>windowCp,floorScore,ticketPower)};const weightedCandidates=candidates.map(candidate=>({candidate:candidate,tickets:(M
    ath.max(0,candidate.score-floorScore)/windowCp)**ticketPower))).filter(({tickets:tickets})=>tickets>0),totalTickets=weigh
    tedCandidates.reduce((total,{tickets:tickets})=>total+tickets,0);if(totalTickets<=0)return{selected:best,debug:createDete
    rministicDebug(candidates,best,best>windowCp,floorScore,ticketPower)};let draw=Math.random()*totalTickets,selected=best;f
    or(const{candidate:candidate,tickets:tickets}of weightedCandidates)if(draw=tickets,draw<=0){selected=candidate;break}ret
    urn{selected:selected,debug:createWeightedDebug(candidates,best,selected,weightedCandidates>windowCp,floorScore,ticketPow

```

```

er, totalTickets)}}(candidates, ctx.settings, principal); return {selected: selection.selected, principal: principal, debug: select
ion.debug}}function shouldStopAfterSoftLimit(chess, rootMoves, result, previousPrincipal, depth, elapsedMs, settings){if("hard"
=== settings.preset || void 0 === settings.softTimeLimitMs || elapsedMs < settings.softTimeLimitMs || depth < 2) return 1; if(chess.isCh
eck()) || result.principal.score < 0) return 1; if(null === previousPrincipal || !isSameMoveCommand(previousPrincipal.move, result.pr
incipal.move) || Math.abs(previousPrincipal.score - result.principal.score) > 90) return 1; const scoreGap = function() {return rootScoreGap(
debug)} {const [bestCandidate, secondCandidate] = debug.candidates; return bestCandidate & secondCandidate ? bestCandidate.score - se
condCandidate.score : 1e7} (result.debug); return (scoreGap < 120 || function() {return tacticalMoveRatio(moves)} {return 0 === moves.length ? 0 :
moves.filter(isTacticalMove).length / moves.length} (rootMoves) > .35 & scoreGap < 220)}function negamax(chess, depth, alpha, beta,
ply, ctx){if(checkTime(ctx), chess.isCheckmate() || chess.isDraw()) return terminalScore(chess, ply); if(depth <= 0) return ctx.set
tings.quiescence ? quiescence(chess, alpha, beta, ply, ctx) : evaluatePosition(chess); let bestScore = -1e7, currentAlpha = alpha; const
legalMoves = orderMoves(chess.moves({verbose: !0}), null); for(const move of legalMoves){chess.move(toMoveCommand(move)); cons
t score = negamax(chess, depth - 1, -beta, -currentAlpha, ply + 1, ctx); if(chess.undo(), bestScore = Math.max(bestScore, score), current
Alpha = Math.max(currentAlpha, score), currentAlpha >= beta) break} return bestScore}function quiescence(chess, alpha, beta, ply, ctx
){if(checkTime(ctx), chess.isCheckmate() || chess.isDraw()) return terminalScore(chess, ply); let currentAlpha = alpha; const stan
dPat = evaluatePosition(chess); if(standPat >= beta) return beta; if(currentAlpha = Math.max(currentAlpha, standPat), ply >= 6) return
currentAlpha; const legalMoves = orderMoves(chess.moves({verbose: !0}), null), tacticalMoves = chess.isCheck() ? legalMoves : legalMo
ves.filter(isTacticalMove); for(const move of tacticalMoves){chess.move(toMoveCommand(move)); const score = -quiescence(chess
, -beta, -currentAlpha, ply + 1, ctx); if(chess.undo(), score >= beta) return beta; currentAlpha = Math.max(currentAlpha, score)} return
currentAlpha}function evaluatePosition(chess){const board = chess.board(), pawns = [], bishops = {w: 0, b: 0}, kings = {}; let whiteScor
e = 0, blackScore = 0, nonPawnMaterial = 0; for(const row of board) for(const pieceOnSquare of row){if(null === pieceOnSquare) continu
e; const {color: color, type: type, square: square} = pieceOnSquare, pieceScore = PIECE_VALUES[type] + pieceSquareScore(type, color, squa
re); "w" === color ? whiteScore += pieceScore : blackScore += pieceScore, "p" === type ? pawns.push({color: color, file: fileIndex(square), r
ank: rankNumber(square)}) : "k" === type & (nonPawnMaterial += PIECE_VALUES[type]), "b" === type & (bishops[color] += 1), "k" === type & (k
ings[color] = square)} whiteScore += bishops.w >= 2 ? 35 : 0, blackScore += bishops.b >= 2 ? 35 : 0, whiteScore += pawnStructureScore(pawns, "w"),
blackScore += pawnStructureScore(pawns, "b"), whiteScore += developmentScore(board, "w"), blackScore += developmentScore(board, "b"),
whiteScore += kingSafetyScore(kings.w, pawns, "w", nonPawnMaterial), blackScore += kingSafetyScore(kings.b, pawns, "b", nonPawnMat
erial); let score = (whiteScore - blackScore) * ("w" === chess.turn() ? 1 : -1); return score += 2 * chess.moves().length, chess.isCheck() &
(score <= 45), score}function pieceSquareScore(type, color, square){const file2 = fileIndex(square), relativeRank = "w" === color ? ran
kNumber(square) : 9 - rankNumber(square), centerDistance = Math.abs(file2 - 3.5) + Math.abs(rankNumber(square) - 4.5); switch(type){cas
e "p": return 7 * relativeRank + (file2 >= 2 & file2 <= 5 ? 8 : 0); case "n": return 42 - 12 * centerDistance - (1 === relativeRank ? 12 : 0); case "b": r
eturn 30 - 7 * centerDistance; case "r": return (relativeRank >= 7 ? 18 : 0) + (0 === file2 ? 7 : file2 > 4 ? 0); case "q": return 14 - 3 * centerDista
nce; case "k": return relativeRank >= 7 ? -18 : 0}}function pawnStructureScore(pawns, color){const ownPawns = pawns.filter(pawn => pawn
.color === color), fileCounts = Array.from({length: 8}, () => 0); let score = 0; for(const pawn of ownPawns) fileCounts[pawn.file] += 1; f
or(const pawn of ownPawns){const relativeRank = "w" === color ? pawn.rank : 9 - pawn.rank; fileCounts[pawn.file] > 1 & (score += 14), 0 ===
(fileCounts[pawn.file] - 1) ? 0 & 0 === (fileCounts[pawn.file] + 1) ? 0 & (score += 10), isPassedPawn(pawn, pawns) & (score += (relativeRa
nk - 1) * (relativeRank - 1) * 9)} return score}function isPassedPawn(pawn, pawns){const enemyColor = function() {return oppositeColor(color)} {r
eturn "w" === color ? "b" : "w"} (pawn.color); return !pawns.some(candidate => !(candidate.color === enemyColor || Math.abs(candidate.fil
e - pawn.file) > 1) & ("w" === pawn.color ? candidate.rank > pawn.rank : candidate.rank < pawn.rank))}function developmentScore(board, co
lor){const homeRank = "w" === color ? "1" : "8", knightHomeSquares = "w" === color ? new Set(["b1", "g1"]) : new Set(["b8", "g8"]), bishopHom
eSquares = "w" === color ? new Set(["c1", "f1"]) : new Set(["c8", "f8"]); let score = 0; for(const row of board) for(const piece of row)
null === piece & piece.color === color & ("n" === piece.type || knightHomeSquares.has(piece.square) || (score += 18), "b" === piece.type ||
bishopHomeSquares.has(piece.square) || (score += 16), "q" === piece.type & piece.square.endsWith(homeRank) & hasMovedMinorPieces(b
oard, color) & (score += 12)); return score}function hasMovedMinorPieces(board, color){const homeSquares = "w" === color ? new Set(["
b1", "c1", "f1", "g1"]) : new Set(["b8", "c8", "f8", "g8"]); for(const row of board) for(const piece of row) if(null === piece & piece.
color === color & ("n" === piece.type || "b" === piece.type) & homeSquares.has(piece.square)) return 1; return 0}function kingSafetyS
core(kingSquare, pawns, color, nonPawnMaterial){if(!kingSquare || nonPawnMaterial < 1800) return 0; const kingFile = fileIndex(kingS
quare), kingRank = rankNumber(kingSquare), shieldRank = "w" === color ? kingRank + 1 : kingRank - 1; let score = 2 === kingFile ? 6 : kingFile ?
28 : -18; for(let file2 = kingFile - 1; file2 <= kingFile + 1; file2 += 1) file2 < 0 || file2 >= 8 || (score += pawns.some(pawn => pawn.color === color
& pawn.file === file2 & pawn.rank === shieldRank) ? 14 : -10); return score}function terminalScore(chess, ply){return chess.isCheckm
ate() ? -1e6 + ply : chess.isDraw() ? 0 : evaluatePosition(chess)}function orderMoves(moves, preferredMove){return [... moves].sort((l
eft, right) => moveOrderingScore(right, preferredMove) - moveOrderingScore(left, preferredMove))}function moveOrderingScore(move
, preferredMove){let score = 0; return preferredMove & function() {return isSameMove(move, command)} {return move.from === command.from & move
.to === command.to & (move.promotion ? void 0) === (command.promotion ? void 0)} (move, preferredMove) & (score += 2e6), move.san.incl
udes("#") ? score += 15e5 : move.san.includes("+") & (score += 16e3), move.isCapture() & (score += 12 * (move.captured ? PIECE_VALUES[move
.captured] : PIECE_VALUES[p] - PIECE_VALUES[move.piece])), move.isPromotion() & move.promotion & (score += PIECE_VALUES[move.promot
ion] + 8e3), (move.isKingsideCastle() || move.isQueensideCastle()) & (score += 1500), score += 2 * pieceSquareScore(move.piece, move.co
lor, move.to), score}function checkTime(ctx){if(ctx.nodes += 1, ctx.nodes % 2048 !== 0) return; const now = Date.now(); if(now - ctx.lastP
rogressAt >= 100 & publishSearchProgress(ctx, ctx.progressState, now - ctx.startTime, now), now >= ctx.deadline) throw new SearchTime
out}function publishSearchProgress(ctx, progressState, elapsedMs = Date.now() - ctx.startTime, progressAt = Date.now()){ctx.progre
ssState = progressState, ctx.lastProgressAt = progressAt, ctx.onProgress ? ({kind: "progress", id: ctx.requestId, move: progressState
.move, depthReached: progressState.depthReached, nodes: ctx.nodes, elapsedMs: elapsedMs, score: progressState.score, debug: progres
sState.debug})}function isTacticalMove(move){return move.isCapture() || move.isPromotion() || move.san.includes("+") || move.sa
n.includes("#")}function createDeterministicDebug(candidates, principal, selected, windowCp, floorScore, ticketPower){return {c
andidates: sortCandidatesByScore(candidates).map(candidate => ({move: candidate.move, san: candidate.san, score: candidate.score,
tickets: isSameMoveCommand(candidate.move, selected.move) ? 1 : 0, probability: isSameMoveCommand(candidate.move, selected.move) ? 1
: 0, selected: isSameMoveCommand(candidate.move, selected.move), principal: isSameMoveCommand(candidate.move, principal.move)})
), bestScore: principal.score, windowCp: windowCp, floorScore: floorScore, ticketPower: ticketPower, totalTickets: 1}}function creat
eWeightedDebug(candidates, principal, selected, weightedCandidates, windowCp, floorScore, ticketPower, totalTickets){return {cand
idates: sortCandidatesByScore(candidates).map(candidate => ({const weightedCandidate = weightedCandidates.find(({candidate: weig
hted}) => isSameMoveCommand(weighted.move, candidate.move))), tickets = weightedCandidate ? tickets : 0; return {move: candidate.move
, san: candidate.san, score: candidate.score, tickets: tickets, probability: totalTickets > 0 ? tickets / totalTickets : 0, selected: isSam

```

```

    ↪ eMoveCommand(candidate.move, selected.move), principal: isSameMoveCommand(candidate.move, principal.move) } } }, bestScore: princi
    ↪ pal.score, windowCp: windowCp, floorScore: floorScore, ticketPower: ticketPower, totalTickets: totalTickets } } function · sortCandida
    ↪ tesByScore(candidates) { return [ ... candidates ]. sort( (left, right) => right.score - left.score ) } function · toMoveCommand(move) { retu
    ↪ rn · move.promotion ? { from: move.from, to: move.to, promotion: move.promotion } : { from: move.from, to: move.to } } function · isSameMoveCom
    ↪ mand(left, right) { return · left.from === right.from && left.to === right.to && (left.promotion ?? void 0) === (right.promotion ?? void 0) }
    ↪ function · fileIndex(square) { return · square.charCodeAt(0) - 97 } function · rankNumber(square) { return · Number(square[1]) } function · c
    ↪ lamp(value, min, max) { return · Math.min(max, Math.max(min, value)) } class · SearchTimeout extends · Error { } const · workerScope = globalT
    ↪ his; workerScope.addEventListener("message", event => { const · response = function · searchBestMove(request, onProgress) { const · chess
    ↪ = new · Chess(request.fen), settings = function · normalizeAiSettings(settings) { const · timeLimitMs = clamp(Math.round(settings.timeL
    ↪ imitMs), 500, 1e4), softTimeLimitMs = void 0 === settings.softTimeLimitMs ? void 0 : Math.min(timeLimitMs, clamp(Math.round(settings.
    ↪ softTimeLimitMs), 500, 1e4)); return { preset: settings.preset, maxDepth: (depth = settings.maxDepth, clamp(Math.round(depth), 1, 5)),
    ↪ timeLimitMs: timeLimitMs, ... void 0 === softTimeLimitMs ? { } : { softTimeLimitMs: softTimeLimitMs }, randomness: 30, quiescence: !0 }; var
    ↪ · depth } (request.settings), legalMoves = chess.moves( { verbose: !0 } ), startTime = Date.now(), ctx = { requestId: request.id, settings: se
    ↪ ttings, deadline: startTime + settings.timeLimitMs, startTime: startTime, nodes: 0, onProgress: onProgress, lastProgressAt: startTime
    ↪ , progressState: { move: null, depthReached: 0, score: 0 } }; if (0 === legalMoves.length) return { id: request.id, move: null, depthReached: 0
    ↪ , nodes: 0, elapsedMs: 0, score: terminalScore(chess, 0) }; let · debug, best = null, previousBestMove = null, previousPrincipal = null, depth
    ↪ Reached = 0; for (let · depth = 1; depth ≤ settings.maxDepth; depth += 1) { try { const · result = searchRoot(chess, depth, ctx, previousBestMove
    ↪ ), elapsedMs = Date.now() - startTime; if (best = result.selected, debug = result.debug, previousBestMove = result.principal.move, depthR
    ↪ eached = depth, publishSearchProgress(ctx, { move: best.move, depthReached: depthReached, score: best.score, debug: debug }, elapsedMs)
    ↪ , shouldStopAfterSoftLimit(chess, legalMoves, result, previousPrincipal, depth, elapsedMs, settings)) break; previousPrincipal = res
    ↪ ult.principal } catch (error) { if (error instanceof · SearchTimeout) break; throw · error } if (Date.now() ≥ ctx.deadline) break } return · b
    ↪ est ?? = function · chooseFallbackMove(chess) { const [move] = orderMoves(chess.moves( { verbose: !0 } ), null); if (!move) return { move: { fro
    ↪ m: "a1", to: "a1" }, score: terminalScore(chess, 0) }; chess.move(toMoveCommand(move)); const · score = -evaluatePosition(chess); return
    ↪ · chess.undo(), { move: toMoveCommand(move), score: score } } (chess), { id: request.id, move: best.move, depthReached: depthReached, node
    ↪ s: ctx.nodes, elapsedMs: Date.now() - startTime, score: best.score, debug: debug } } (event.data, progress => { workerScope.postMessage(p
    ↪ rogress) }); workerScope.postMessage( { ... response, kind: "result" } ) } } (); \n',

951 | · blob =
952 | · 'undefined' ≠ typeof self · &&
953 | · self.Blob · &&
954 | · new · Blob([ (self.URL · || self.webkitURL).revokeObjectURL(self.location.href);', · jsContent ], {
955 | · · type: 'text/javascript; charset=utf-8',
956 | · · });
957 | function · WorkerWrapper(options) {
958 | · let · objURL;
959 | · try {
960 | · · if ( ((objURL = blob · && (self.URL · || self.webkitURL).createObjectURL(blob)), !objURL) ) · throw · '';
961 | · · const · worker = new · Worker(objURL, { name: options?.name });
962 | · · return (
963 | · · · worker.addEventListener('error', ( ) => {
964 | · · · (self.URL · || self.webkitURL).revokeObjectURL(objURL);
965 | · · · } ),
966 | · · worker
967 | · · );
968 | · } catch (e) {
969 | · · return new · Worker('data:text/javascript; charset=utf-8,' + encodeURIComponent(jsContent), {
970 | · · · name: options?.name,
971 | · · · });
972 | · }
973 | }
974 | const · BOARD_FILES = [ 'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h' ],
975 | · BLACK_FILES = [ ... BOARD_FILES ].reverse(),
976 | · WHITE_RANKS = [ 8, 7, 6, 5, 4, 3, 2, 1 ],
977 | · BLACK_RANKS = [ ... WHITE_RANKS ].reverse(),
978 | · PROMOTION_PIECES = [ 'q', 'r', 'b', 'n' ],
979 | · SQUARE_SET = new · Set([
980 | · · 'a8',
981 | · · 'b8',
982 | · · 'c8',
983 | · · 'd8',
984 | · · 'e8',
985 | · · 'f8',
986 | · · 'g8',
987 | · · 'h8',
988 | · · 'a7',
989 | · · 'b7',
990 | · · 'c7',
991 | · · 'd7',
992 | · · 'e7',
993 | · · 'f7',
994 | · · 'g7',
995 | · · 'h7',

```

```

996 |     'a6',
997 |     'b6',
998 |     'c6',
999 |     'd6',
1000 |     'e6',
1001 |     'f6',
1002 |     'g6',
1003 |     'h6',
1004 |     'a5',
1005 |     'b5',
1006 |     'c5',
1007 |     'd5',
1008 |     'e5',
1009 |     'f5',
1010 |     'g5',
1011 |     'h5',
1012 |     'a4',
1013 |     'b4',
1014 |     'c4',
1015 |     'd4',
1016 |     'e4',
1017 |     'f4',
1018 |     'g4',
1019 |     'h4',
1020 |     'a3',
1021 |     'b3',
1022 |     'c3',
1023 |     'd3',
1024 |     'e3',
1025 |     'f3',
1026 |     'g3',
1027 |     'h3',
1028 |     'a2',
1029 |     'b2',
1030 |     'c2',
1031 |     'd2',
1032 |     'e2',
1033 |     'f2',
1034 |     'g2',
1035 |     'h2',
1036 |     'a1',
1037 |     'b1',
1038 |     'c1',
1039 |     'd1',
1040 |     'e1',
1041 |     'f1',
1042 |     'g1',
1043 |     'h1',
1044 |   ]),
1045 |   CLOCK_PRESETS:={
1046 |     '1/1':{baseMs:6e4,incrementMs:1e3},
1047 |     '3/2':{baseMs:18e4,incrementMs:2e3},
1048 |     '5/3':{baseMs:3e5,incrementMs:3e3},
1049 |     '10/5':{baseMs:6e5,incrementMs:5e3},
1050 |     '15/10':{baseMs:9e5,incrementMs:1e4},
1051 |   },
1052 |   PIECE_GLYPHS:={
1053 |     w:{p:'△',n:'△',b:'▲',r:'▣',q:'⊕',k:'♔'},
1054 |     b:{p:'▲',n:'▲',b:'♙',r:'♚',q:'♛',k:'♜'},
1055 |   };
1056 |   function requireElement(selector, root){
1057 |     const element = root.querySelector(selector);
1058 |     if (!element) throw new Error(`Missing element: ${selector}`);
1059 |     return element;
1060 |   }
1061 |   function createCoordinate(type, label){
1062 |     const coordinate = document.createElement('span');
1063 |     return (
1064 |       (coordinate.className = `coordinate coordinate-${type}`),
1065 |       (coordinate.textContent = label),
1066 |       coordinate

```

```

1067 | |.);
1068 | }
1069 | function toSquare(file2, rank2) {
1070 | |return `${file2}${rank2}`;
1071 | }
1072 | function createClockState(preset) {
1073 | |const baseMs = CLOCK_PRESETS[preset].baseMs;
1074 | |return {w: baseMs, b: baseMs};
1075 | }
1076 | function colorName(color) {
1077 | |return 'w' === color ? '백' : '흑';
1078 | }
1079 | function presetName(preset) {
1080 | |switch (preset) {
1081 | | |case 'easy':
1082 | | | |return '쉬움';
1083 | | |case 'normal':
1084 | | | |return '보통';
1085 | | |case 'hard':
1086 | | | |return '어려움';
1087 | | }
1088 | }
1089 | function formatSeconds(milliseconds) {
1090 | |return `${(milliseconds / 1e3).toFixed(milliseconds % 1e3 === 0 ? 0 : 2)}초`;
1091 | }
1092 | function formatDebugScore(score) {
1093 | |return Math.abs(score) >= 9e5
1094 | | | ? score > 0
1095 | | | | ? '+mate-ish'
1096 | | | | : '-mate-ish'
1097 | | | : score > 0
1098 | | | | ? `+${Math.round(score)}`
1099 | | | | : `-${String(Math.round(score))}`;
1100 | }
1101 | function oppositeColor(color) {
1102 | |return 'w' === color ? 'b' : 'w';
1103 | }
1104 | function getSquareLabel(square, piece) {
1105 | |return piece ? `${square}-${colorName(piece.color)}-${pieceName(piece.type)}` : `${square}-빈칸`;
1106 | }
1107 | function pieceName(piece) {
1108 | |switch (piece) {
1109 | | |case 'p':
1110 | | | |return '폰';
1111 | | |case 'n':
1112 | | | |return '나이트';
1113 | | |case 'b':
1114 | | | |return '비숍';
1115 | | |case 'r':
1116 | | | |return '룩';
1117 | | |case 'q':
1118 | | | |return '퀸';
1119 | | |case 'k':
1120 | | | |return '킹';
1121 | | }
1122 | }
1123 | !(function createChessApp() {
1124 | |const root = requireElement('#chess-app', document),
1125 | |boardElement = requireElement('#chess-board', root),
1126 | |colorControlsElement = requireElement('#color-controls', root),
1127 | |aiControlsElement = requireElement('#ai-controls', root),
1128 | |statusElement = requireElement('#game-status', root),
1129 | |turnMetaElement = requireElement('#turn-meta', root),
1130 | |modeMetaElement = requireElement('#mode-meta', root),
1131 | |moveMetaElement = requireElement('#move-meta', root),
1132 | |aiCandidatePanelElement = requireElement('#ai-candidate-panel', root),
1133 | |aiCandidateMetaElement = requireElement('#ai-candidate-meta', root),
1134 | |aiInfoStatusElement = requireElement('#ai-info-status', root),
1135 | |aiInfoPresetElement = requireElement('#ai-info-preset', root),
1136 | |aiInfoHumanElement = requireElement('#ai-info-human', root),
1137 | |aiInfoDepthElement = requireElement('#ai-info-depth', root),

```

```

1138 |     aiInfoNodesElement := requireElement('#ai-info-nodes', root),
1139 |     aiInfoScoreElement := requireElement('#ai-info-score', root),
1140 |     aiCandidateListElement := requireElement('#ai-candidate-list', root),
1141 |     moveListElement := requireElement('#move-list', root),
1142 |     clockDisplayElement := requireElement('#clock-display', root),
1143 |     whiteClockElement := requireElement('#white-clock', root),
1144 |     blackClockElement := requireElement('#black-clock', root),
1145 |     whiteClockTimeElement := requireElement('#white-clock-time', root),
1146 |     blackClockTimeElement := requireElement('#black-clock-time', root),
1147 |     newGameButton := requireElement('#new-game-button', root),
1148 |     undoButton := requireElement('#undo-button', root),
1149 |     aiDepthInput := requireElement('#ai-depth-input', root),
1150 |     aiDepthValue := requireElement('#ai-depth-value', root),
1151 |     aiTimeInput := requireElement('#ai-time-input', root),
1152 |     aiTimeValue := requireElement('#ai-time-value', root),
1153 |     promotionOverlay := requireElement('#promotion-overlay', root),
1154 |     promotionChoices := requireElement('#promotion-choices', root),
1155 |     modeButtons := Array.from(root.querySelectorAll('[data-mode]')),
1156 |     colorButtons := Array.from(root.querySelectorAll('[data-color]')),
1157 |     presetButtons := Array.from(root.querySelectorAll('[data-preset]')),
1158 |     clockPresetButtons := Array.from(root.querySelectorAll('[data-clock-preset]')),
1159 |     game := new Chess();
1160 |     let mode = 'local',
1161 |         clockPreset = '10/5',
1162 |         clockMs := createClockState(clockPreset),
1163 |         activeClockColor := null,
1164 |         lastClockTick := Date.now(),
1165 |         clockTimerId := null,
1166 |         timedOutColor := null,
1167 |         humanColor = 'w',
1168 |         selectedSquare := null,
1169 |         legalMoves := [],
1170 |         pendingPromotion := null,
1171 |         aiThinking := !1,
1172 |         aiTimerId := null,
1173 |         aiWorker := null,
1174 |         aiRequestId = 0,
1175 |         activeAiRequestId := null,
1176 |         activeAiFen := null,
1177 |         aiSettings = { ... DEFAULT_AI_SETTINGS },
1178 |         aiProgress := null;
1179 |     function selectSquare(square) {
1180 |         ((selectedSquare := square),
1181 |         ((legalMoves := game.moves({ verbose: !0, square: square })),
1182 |         render());
1183 |     }
1184 |     function playMove(moveCommand) {
1185 |         if (!commitMove(moveCommand)) return (clearSelection(), void render());
1186 |         ((pendingPromotion := null), clearSelection(), render(), scheduleAiIfNeeded());
1187 |     }
1188 |     function resetGame() {
1189 |         (cancelAiMove(),
1190 |         game.reset(),
1191 |         (pendingPromotion := null),
1192 |         clearSelection(),
1193 |         resetClockState(),
1194 |         restartClockForCurrentTurn(),
1195 |         render(),
1196 |         scheduleAiIfNeeded());
1197 |     }
1198 |     function scheduleAiIfNeeded() {
1199 |         'ai' === mode ||
1200 |         game.isGameOver() ||
1201 |         null === timedOutColor ||
1202 |         game.turn() === humanColor ||
1203 |         aiThinking ||
1204 |         pendingPromotion ||
1205 |         ((aiThinking := !0),
1206 |         (aiProgress := null),
1207 |         render(),
1208 |         (aiTimerId := window.setTimeout(() => {

```

```

1209 | | | | | ((aiTimerId = null),
1210 | | | | | (function startAiSearch() {
1211 | | | | |   aiWorker?.terminate();
1212 | | | | |   const requestId = aiRequestId + 1,
1213 | | | | |   fen = game.fen();
1214 | | | | |   ((aiRequestId = requestId),
1215 | | | | |   (activeAiRequestId = requestId),
1216 | | | | |   (activeAiFen = fen),
1217 | | | | |   (aiWorker = new WorkerWrapper()),
1218 | | | | |   aiWorker.addEventListener('message', (event) => {
1219 | | | | |     ! (function handleAiMessage(message) {
1220 | | | | |       if (
1221 | | | | |         message.id === activeAiRequestId &&
1222 | | | | |         null !== activeAiFen &&
1223 | | | | |         game.fen() === activeAiFen
1224 | | | | |       ) {
1225 | | | | |         if (((aiProgress = message), 'progress' === message.kind))
1226 | | | | |           return (renderStatus(), void renderAiCandidateDebug());
1227 | | | | |         ((aiThinking = !1),
1228 | | | | |         (activeAiRequestId = null),
1229 | | | | |         (activeAiFen = null),
1230 | | | | |         aiWorker?.terminate(),
1231 | | | | |         (aiWorker = null),
1232 | | | | |         message.move && commitMove(message.move),
1233 | | | | |         clearSelection(),
1234 | | | | |         render());
1235 | | | | |       }
1236 | | | | |     })(event.data);
1237 | | | | |   }),
1238 | | | | |   aiWorker.addEventListener('error', () => {
1239 | | | | |     activeAiRequestId === requestId &&
1240 | | | | |     ((aiThinking = !1),
1241 | | | | |     (activeAiRequestId = null),
1242 | | | | |     (activeAiFen = null),
1243 | | | | |     aiWorker?.terminate(),
1244 | | | | |     (aiWorker = null),
1245 | | | | |     render());
1246 | | | | |   })),
1247 | | | | |   aiWorker.postMessage({
1248 | | | | |     id: requestId,
1249 | | | | |     fen: fen,
1250 | | | | |     settings: resolveAiSettingsForSearch(),
1251 | | | | |   }));
1252 | | | | | }());
1253 | | | | | }, 180));
1254 | | | }
1255 | | function cancelAiMove() {
1256 | |   ((aiRequestId += 1),
1257 | |   (activeAiRequestId = null),
1258 | |   (activeAiFen = null),
1259 | |   (aiProgress = null),
1260 | |   null !== aiTimerId && (window.clearTimeout(aiTimerId), (aiTimerId = null)),
1261 | |   aiWorker && (aiWorker.terminate(), (aiWorker = null)),
1262 | |   (aiThinking = !1));
1263 | | }
1264 | | function commitMove(moveCommand) {
1265 | |   const movingColor = game.turn();
1266 | |   if (!flushClockElapsed()) return !1;
1267 | |   try {
1268 | |     game.move(moveCommand);
1269 | |   } catch {
1270 | |     return !1;
1271 | |   }
1272 | |   return (
1273 | |     (clockMs[movingColor] += CLOCK_PRESETS[clockPreset].incrementMs),
1274 | |     restartClockForCurrentTurn(),
1275 | |     !0
1276 | |   );
1277 | | }
1278 | | function updateAiSettings(settings) {
1279 | |   ((aiSettings = normalizeAiSettings(settings)),

```

```

1280 | | | | | aiThinking && cancelAiMove(),
1281 | | | | | render(),
1282 | | | | | scheduleAiIfNeeded());
1283 | | | }
1284 | | | function resolveAiSettingsForSearch() {
1285 | | | | | const settings = normalizeAiSettings(aiSettings);
1286 | | | | | if ('hard' !== settings.preset) return settings;
1287 | | | | | const aiColor = game.turn(),
1288 | | | | | remainingMs = clockMs[aiColor],
1289 | | | | | targetMs = remainingMs / 30 + 0.8 * CLOCK_PRESETS[clockPreset].incrementMs,
1290 | | | | | remainingSearchBudgetMs = Math.max(500, remainingMs - 250),
1291 | | | | | softTimeLimitMs = Math.min(Math.max(targetMs, 800), 6e3, remainingSearchBudgetMs),
1292 | | | | | timeLimitMs = Math.min(Math.max(1.8 * softTimeLimitMs, 1200), 1e4, remainingSearchBudgetMs);
1293 | | | | | return normalizeAiSettings({
1294 | | | | | | | ... settings,
1295 | | | | | | | softTimeLimitMs: softTimeLimitMs,
1296 | | | | | | | timeLimitMs: timeLimitMs,
1297 | | | | | });
1298 | | | }
1299 | | | function resetClockState() {
1300 | | | | | ((clockMs = createClockState(clockPreset)),
1301 | | | | | (timedOutColor = null),
1302 | | | | | (activeClockColor = null),
1303 | | | | | (lastClockTick = Date.now()));
1304 | | | }
1305 | | | function restartClockForCurrentTurn() {
1306 | | | | | if ((clearClockTimer(), game.isGameOver() || null !== timedOutColor))
1307 | | | | | return ((activeClockColor = null), void renderClocks());
1308 | | | | | ((activeClockColor = game.turn()),
1309 | | | | | (lastClockTick = Date.now()),
1310 | | | | | (clockTimerId = window.setInterval(tickClock, 100)),
1311 | | | | | renderClocks());
1312 | | | }
1313 | | | function clearClockTimer() {
1314 | | | | | null !== clockTimerId && (window.clearInterval(clockTimerId), (clockTimerId = null));
1315 | | | }
1316 | | | function tickClock() {
1317 | | | | | flushClockElapsed() && renderClocks();
1318 | | | }
1319 | | | function flushClockElapsed() {
1320 | | | | | if (null === activeClockColor || null !== timedOutColor || game.isGameOver()) return !0;
1321 | | | | | const now = Date.now(),
1322 | | | | | elapsedMs = Math.max(0, now - lastClockTick);
1323 | | | | | return (
1324 | | | | | | | (lastClockTick = now),
1325 | | | | | | | 0 === elapsedMs ||
1326 | | | | | | | ((clockMs[activeClockColor] = Math.max(0, clockMs[activeClockColor] - elapsedMs)),
1327 | | | | | | | 0 !== clockMs[activeClockColor] ||
1328 | | | | | | | ((function handleClockTimeout(color) {
1329 | | | | | | | | | ((timedOutColor = color),
1330 | | | | | | | | | (activeClockColor = null),
1331 | | | | | | | | | (clockMs[color] = 0),
1332 | | | | | | | | | clearClockTimer(),
1333 | | | | | | | | | cancelAiMove(),
1334 | | | | | | | | | clearSelection(),
1335 | | | | | | | | | render();
1336 | | | | | | | | | })(activeClockColor),
1337 | | | | | | | !1))
1338 | | | | | );
1339 | | | }
1340 | | | function clearSelection() {
1341 | | | | | ((selectedSquare = null), (legalMoves = []));
1342 | | | }
1343 | | | function render() {
1344 | | | | | (!function renderControls() {
1345 | | | | | | | (modeButtons.forEach((button) => {
1346 | | | | | | | | | const isActive = button.dataset.mode === mode;
1347 | | | | | | | | | (button.classList.toggle('is-active', isActive),
1348 | | | | | | | | | button.setAttribute('aria-pressed', String(isActive)));
1349 | | | | | | | })),
1350 | | | | | | | colorButtons.forEach((button) => {

```



```

1351 |         const isActive = button.dataset.color === humanColor;
1352 |         (button.classList.toggle('is-active', isActive),
1353 |         button.setAttribute('aria-pressed', String(isActive)));
1354 |     },
1355 |     (colorControlsElement.hidden = 'ai' !== mode),
1356 |     (aiControlsElement.hidden = 'ai' !== mode),
1357 |     (function renderAiSettingsControls() {
1358 |         (presetButtons.forEach((button) => {
1359 |             const isActive = button.dataset.preset === aiSettings.preset;
1360 |             (button.classList.toggle('is-active', isActive),
1361 |             button.setAttribute('aria-pressed', String(isActive)));
1362 |         })),
1363 |         (aiDepthInput.value = String(aiSettings.maxDepth)),
1364 |         (aiDepthValue.textContent = `${aiSettings.maxDepth}`),
1365 |         (aiTimeInput.value = String(aiSettings.timeLimitMs)),
1366 |         (aiTimeInput.disabled = 'hard' === aiSettings.preset),
1367 |         (aiTimeValue.textContent =
1368 |         'hard' === aiSettings.preset ? '자동' : `${formatSeconds(aiSettings.timeLimitMs)}`));
1369 |     })(),
1370 |     (function renderClockSettingsControls() {
1371 |         clockPresetButtons.forEach((button) => {
1372 |             const isActive = button.dataset.clockPreset === clockPreset;
1373 |             (button.classList.toggle('is-active', isActive),
1374 |             button.setAttribute('aria-pressed', String(isActive)));
1375 |         });
1376 |     })(),
1377 |     (undoButton.disabled = 0 === game.history().length));
1378 | })(),
1379 | renderClocks(),
1380 | (function renderBoard() {
1381 |     const fragment = document.createDocumentFragment(),
1382 |     orientation = getBoardOrientation(),
1383 |     files = 'w' === orientation ? BOARD_FILES : BLACK_FILES,
1384 |     ranks = 'w' === orientation ? WHITE_RANKS : BLACK_RANKS,
1385 |     lastMove = (function getLastMove() {
1386 |         return game.history({ verbose: !0 }).at(-1) ?? null;
1387 |     })();
1388 |     boardElement.classList.toggle('is-locked', !canUseBoard());
1389 |     for (const rank2 of ranks)
1390 |         for (const file2 of files) {
1391 |             const square = toSquare(file2, rank2),
1392 |             piece = game.get(square),
1393 |             button = document.createElement('button');
1394 |             if (
1395 |                 ((button.type = 'button'),
1396 |                 (button.className = getSquareClasses(square, lastMove)),
1397 |                 (button.dataset.square = square),
1398 |                 (button.disabled = !canUseBoard()),
1399 |                 button.setAttribute('role', 'gridcell'),
1400 |                 button.setAttribute('aria-label', getSquareLabel(square, piece)),
1401 |                 file2 === files[0] && button.appendChild(createCoordinate('rank', String(rank2))),
1402 |                 rank2 === ranks[ranks.length - 1] &&
1403 |                 button.appendChild(createCoordinate('file', file2)),
1404 |                 piece)
1405 |             ) {
1406 |                 const pieceElement = document.createElement('span');
1407 |                 ((pieceElement.className = 'piece'),
1408 |                 (pieceElement.textContent = PIECE_GLYPHS[piece.color][piece.type]),
1409 |                 button.appendChild(pieceElement));
1410 |             }
1411 |             fragment.appendChild(button);
1412 |         }
1413 |     boardElement.replaceChildren(fragment);
1414 | })(),
1415 | renderStatus(),
1416 | renderAiCandidateDebug(),
1417 | (function renderMoveList() {
1418 |     const history = game.history({ verbose: !0 }),
1419 |     fragment = document.createDocumentFragment();
1420 |     if (0 === history.length) {
1421 |         const empty = document.createElement('p');

```

```

1422 |         return (
1423 |             (empty.className = 'move-empty'),
1424 |             (empty.textContent = '아직 둔 수가 없습니다.'),
1425 |             fragment.appendChild(empty),
1426 |             void moveListElement.replaceChildren(fragment)
1427 |         );
1428 |     }
1429 |     for (let index = 0; index < history.length; index += 2) {
1430 |         const row = document.createElement('div'),
1431 |             moveNumber = document.createElement('span'),
1432 |             whiteMove = document.createElement('span'),
1433 |             blackMove = document.createElement('span');
1434 |         ((row.className = 'move-row'),
1435 |         (moveNumber.className = 'move-number'),
1436 |         (whiteMove.className = 'move-san'),
1437 |         (blackMove.className = 'move-san'),
1438 |         (moveNumber.textContent = `${Math.floor(index / 2) + 1}.`),
1439 |         (whiteMove.textContent = history[index]?.san ?? ''),
1440 |         (blackMove.textContent = history[index + 1]?.san ?? '')),
1441 |         row.append(moveNumber, whiteMove, blackMove),
1442 |         fragment.appendChild(row));
1443 |     }
1444 |     (moveListElement.replaceChildren(fragment),
1445 |     (moveListElement.scrollTop = moveListElement.scrollHeight));
1446 |     })(),
1447 |     (function renderPromotionDialog() {
1448 |         if (
1449 |             ((promotionOverlay.hidden = null === pendingPromotion),
1450 |             promotionChoices.replaceChildren(),
1451 |             pendingPromotion)
1452 |         )
1453 |             for (const piece of PROMOTION_PIECES) {
1454 |                 const button = document.createElement('button');
1455 |                 ((button.className = 'promotion-button'),
1456 |                 (button.type = 'button'),
1457 |                 (button.dataset.promotion = piece),
1458 |                 button.setAttribute('aria-label', `${pieceName(piece)} 승격`),
1459 |                 (button.textContent = PIECE_GLYPHS[game.turn()][piece]),
1460 |                 promotionChoices.appendChild(button));
1461 |             }
1462 |         })();
1463 | }
1464 | function renderClocks() {
1465 |     const bottomColor = getBoardOrientation(),
1466 |         topClockElement = 'w' === oppositeColor(bottomColor) ? whiteClockElement : blackClockElement,
1467 |         bottomClockElement = 'w' === bottomColor ? whiteClockElement : blackClockElement;
1468 |     ((clockDisplayElement.children[0] === topClockElement &&
1469 |     clockDisplayElement.children[1] === bottomClockElement) ? ||
1470 |     clockDisplayElement.replaceChildren(topClockElement, bottomClockElement),
1471 |     renderClockPosition('w', bottomColor),
1472 |     renderClockPosition('b', bottomColor),
1473 |     renderClock('w', whiteClockElement, whiteClockTimeElement),
1474 |     renderClock('b', blackClockElement, blackClockTimeElement));
1475 | }
1476 | function renderClockPosition(color, bottomColor) {
1477 |     const clockElement = 'w' === color ? whiteClockElement : blackClockElement,
1478 |         isBottom = color === bottomColor;
1479 |     (clockElement.classList.toggle('is-bottom', isBottom),
1480 |     clockElement.classList.toggle('is-top', !isBottom));
1481 | }
1482 | function renderClock(color, clockElement, timeElement) {
1483 |     const remainingMs = clockMs[color],
1484 |         isActive = activeClockColor === color && null === timedOutColor,
1485 |         isTimedOut = timedOutColor === color;
1486 |     (clockElement.classList.toggle('is-active', isActive),
1487 |     clockElement.classList.toggle('is-warning', remainingMs ≤ 3e4),
1488 |     clockElement.classList.toggle('is-danger', remainingMs ≤ 1e4),
1489 |     clockElement.classList.toggle('is-timeout', isTimedOut),
1490 |     (timeElement.textContent = (function formatClockTime(milliseconds) {
1491 |         const safeMilliseconds = Math.max(0, milliseconds);
1492 |         if (safeMilliseconds < 1e4)

```

```

1493 |         return `${(safeMilliseconds / 1e3).toFixed(1).padStart(4, '0')}`;
1494 |         const totalSeconds = Math.ceil(safeMilliseconds / 1e3);
1495 |         return `${Math.floor(totalSeconds / 60)}:${String(totalSeconds % 60).padStart(2, '0')}`;
1496 |     })(remainingMs));
1497 | }
1498 | function renderStatus() {
1499 |     const historyLength = game.history().length;
1500 |     ((statusElement.textContent = (function getStatusText() {
1501 |         return game.isCheckmate()
1502 |             ? `${colorName(oppositeColor(game.turn()))} 체크메이트 승리`
1503 |             : game.isStalemate()
1504 |                 ? '스테일메이트 무승부'
1505 |                 : game.isInsufficientMaterial()
1506 |                     ? '기물 부족 무승부'
1507 |                     : game.isThreefoldRepetition()
1508 |                         ? '3회 반복 무승부'
1509 |                         : game.isDrawByFiftyMoves()
1510 |                             ? '50수 규칙 무승부'
1511 |                             : game.isDraw()
1512 |                                 ? '무승부'
1513 |                                 : timedOutColor
1514 |                                     ? [
1515 |                                         `${colorName(timedOutColor)} 시간 초과`,
1516 |                                         `${colorName(oppositeColor(timedOutColor))} 승리`,
1517 |                                     ].join('...')
1518 |                                     : aiThinking
1519 |                                         ? 'AI 계산 중'
1520 |                                         : game.isCheck()
1521 |                                             ? `${colorName(game.turn())} 체크`
1522 |                                             : `${colorName(game.turn())} 차례`;
1523 |     })()),
1524 |     (turnMetaElement.textContent = colorName(game.turn())),
1525 |     (modeMetaElement.textContent =
1526 |         'local' === mode
1527 |             ? '로컬 2인'
1528 |             : ['AI 상대', `인간-${colorName(humanColor)}`, presetName(aiSettings.preset)].join(
1529 |                 '...',
1530 |             )),
1531 |     (moveMetaElement.textContent = [`${game.moveNumber()}수`, `${historyLength} half-move`].join(
1532 |         '...',
1533 |     )),
1534 | }
1535 | function renderAiCandidateDebug() {
1536 |     if ('ai' !== mode)
1537 |         return (
1538 |             (aiCandidatePanelElement.hidden = !0),
1539 |             (aiCandidateMetaElement.textContent = ''),
1540 |             (aiInfoStatusElement.textContent = ''),
1541 |             (aiInfoPresetElement.textContent = ''),
1542 |             (aiInfoHumanElement.textContent = ''),
1543 |             (aiInfoDepthElement.textContent = ''),
1544 |             (aiInfoNodesElement.textContent = ''),
1545 |             (aiInfoScoreElement.textContent = ''),
1546 |             void aiCandidateListElement.replaceChildren()
1547 |         );
1548 |     const debug = aiProgress?.debug;
1549 |     if (
1550 |         ((aiCandidatePanelElement.hidden = !1),
1551 |         (aiCandidateMetaElement.textContent = aiProgress
1552 |             ? `경과 ${formatSeconds(aiProgress.elapsedMs)}`
1553 |             : ''),
1554 |         (aiInfoStatusElement.textContent = (function getAiInfoStatusText() {
1555 |             return game.isGameOver() || null !== timedOutColor
1556 |                 ? '대국 종료'
1557 |                 : aiThinking
1558 |                     ? aiProgress
1559 |                     : '계산 중'
1560 |                     : '계산 준비 중'
1561 |                     : game.turn() === humanColor
1562 |                         ? '대기 중'
1563 |                         : '응수 대기';

```

```

1564 | | | | |.}()),
1565 | | | | |(aiInfoPresetElement.textContent = presetName(aiSettings.preset)),
1566 | | | | |(aiInfoHumanElement.textContent = colorName(humanColor)),
1567 | | | | |(aiInfoDepthElement.textContent = aiProgress ? String(aiProgress.depthReached) : '-'),
1568 | | | | |(aiInfoNodesElement.textContent = aiProgress
1569 | | | | | ? ` ${ (function formatNodes(nodes) {
1570 | | | | |     return nodes >= 1e6
1571 | | | | |     ? ` ${ (nodes / 1e6).toFixed(1)}M`
1572 | | | | |     : nodes >= 1e3
1573 | | | | |     ? ` ${Math.round(nodes / 1e3)}k`
1574 | | | | |     : String(nodes);
1575 | | | | | })(aiProgress.nodes) } .nodes`
1576 | | | | | : '-'),
1577 | | | | |(aiInfoScoreElement.textContent = aiProgress ? formatDebugScore(aiProgress.score) : '-'),
1578 | | | | | !debug || 0 === debug.candidates.length)
1579 | | | | | )
1580 | | | | | return void aiCandidateListElement.replaceChildren();
1581 | | | | | aiCandidateMetaElement.textContent = [
1582 | | | | |     aiCandidateMetaElement.textContent,
1583 | | | | |     `best- ${formatDebugScore(debug.bestScore)}`,
1584 | | | | |     `창- ${Math.round(debug.windowCp)}`,
1585 | | | | |     `p- ${debug.ticketPower.toFixed(2)}`,
1586 | | | | | ].join(' ');
1587 | | | | | const fragment = document.createDocumentFragment();
1588 | | | | | for (const candidate of debug.candidates) fragment.appendChild(createAiCandidateRow(candidate));
1589 | | | | | aiCandidateListElement.replaceChildren(fragment);
1590 | | | | | }
1591 | | | | | function createAiCandidateRow(candidate) {
1592 | | | | |     const row = document.createElement('div');
1593 | | | | |     ((row.className = 'ai-candidate-row'),
1594 | | | | |     row.classList.toggle('is-selected', candidate.selected),
1595 | | | | |     row.classList.toggle('is-principal', candidate.principal),
1596 | | | | |     row.setAttribute('role', 'row'));
1597 | | | | |     const sanCell = createAiCandidateCell('ai-candidate-san', candidate.san);
1598 | | | | |     return (
1599 | | | | |         candidate.selected && sanCell.appendChild(createAiCandidateBadge('SEL')),
1600 | | | | |         candidate.principal && sanCell.appendChild(createAiCandidateBadge('PV')),
1601 | | | | |         row.append(
1602 | | | | |             sanCell,
1603 | | | | |             createAiCandidateCell('ai-candidate-score', formatDebugScore(candidate.score)),
1604 | | | | |             createAiCandidateCell(
1605 | | | | |                 'ai-candidate-weight',
1606 | | | | |                 (function formatDebugWeight(weight) {
1607 | | | | |                     return 0 === weight
1608 | | | | |                     ? '0'
1609 | | | | |                     : weight < 0.001
1610 | | | | |                     ? '<0.001'
1611 | | | | |                     : weight.toFixed(3).replace(/0+$/, '').replace(/\./, '');
1612 | | | | |                 })(candidate.tickets),
1613 | | | | |             ),
1614 | | | | |             createAiCandidateCell(
1615 | | | | |                 'ai-candidate-probability',
1616 | | | | |                 (function formatDebugProbability(probability) {
1617 | | | | |                     const percent = 100 * probability;
1618 | | | | |                     return 0 === percent ? '0%' : percent < 0.1 ? '<0.1%' : ` ${percent.toFixed(1)}%`;
1619 | | | | |                 })(candidate.probability),
1620 | | | | |             ),
1621 | | | | |         ),
1622 | | | | |         row
1623 | | | | |     );
1624 | | | | | }
1625 | | | | | function createAiCandidateCell(className, text) {
1626 | | | | |     const cell = document.createElement('span');
1627 | | | | |     return (
1628 | | | | |         (cell.className = className),
1629 | | | | |         cell.setAttribute('role', 'cell'),
1630 | | | | |         (cell.textContent = text),
1631 | | | | |         cell
1632 | | | | |     );
1633 | | | | | }
1634 | | | | | function createAiCandidateBadge(text) {

```

```

1635 |   const badge = document.createElement('span');
1636 |   return ((badge.className = 'ai-candidate-badge'), (badge.textContent = text), badge);
1637 | }
1638 | function getSquareClasses(square, lastMove) {
1639 |   const classes = ['board-square', 'light' === game.squareColor(square) ? 'is-light' : 'is-dark'],
1640 |     destinationMove = legalMoves.find((move) => move.to === square);
1641 |   return (
1642 |     selectedSquare === square && classes.push('is-selected'),
1643 |     destinationMove && classes.push(destinationMove.isCapture() ? 'is-capture' : 'is-legal'),
1644 |     !lastMove ||
1645 |     (lastMove.from !== square && lastMove.to !== square) ||
1646 |     classes.push('is-last-move'),
1647 |     (function isCheckedKing(square) {
1648 |       const piece = game.get(square);
1649 |       return Boolean(
1650 |         piece && 'k' === piece.type && piece.color === game.turn() && game.isCheck(),
1651 |         );
1652 |     })(square) && classes.push('is-check'),
1653 |     classes.join(' ')
1654 |   );
1655 | }
1656 | function getBoardOrientation() {
1657 |   return 'ai' === mode ? humanColor : game.turn();
1658 | }
1659 | function canUseBoard() {
1660 |   return !(
1661 |     game.isGameOver() ||
1662 |     null !== timedOutColor ||
1663 |     aiThinking ||
1664 |     pendingPromotion ||
1665 |     ('local' !== mode && game.turn() !== humanColor)
1666 |   );
1667 | }
1668 | boardElement.addEventListener('click', (event) => {
1669 |   const button = (function getSquareButton(target, boardElement) {
1670 |     if (!(target instanceof Element)) return null;
1671 |     const button = target.closest('.board-square');
1672 |     return button && boardElement.contains(button) ? button : null;
1673 |   })(event.target, boardElement);
1674 |   button &&
1675 |     (function isSquare(value) {
1676 |       return 'string' === typeof value && SQUARE_SET.has(value);
1677 |     })(button.dataset.square) &&
1678 |     (function handleSquareClick(square) {
1679 |       if (!canUseBoard()) return;
1680 |       const piece = game.get(square);
1681 |       if (selectedSquare) {
1682 |         const movesToDestination = legalMoves.filter((move) => move.to === square);
1683 |         return movesToDestination.length > 0
1684 |           ? movesToDestination.some((move) => move.isPromotion())
1685 |           ? ((pendingPromotion = {
1686 |               from: selectedSquare,
1687 |               to: square,
1688 |               moves: movesToDestination,
1689 |             })),
1690 |           void render()
1691 |           : void playMove({ from: selectedSquare, to: square })
1692 |           : piece ? color === game.turn()
1693 |           ? void selectSquare(square)
1694 |           : (clearSelection(), void render());
1695 |       }
1696 |       piece ? color === game.turn() && selectSquare(square);
1697 |     })(button.dataset.square);
1698 |   },
1699 |   modeButtons.forEach((button) => {
1700 |     button.addEventListener('click', () => {
1701 |       const nextMode = button.dataset.mode;
1702 |       (function isGameMode(value) {
1703 |         return 'local' === value || 'ai' === value;
1704 |       })(nextMode) &&
1705 |       nextMode !== mode &&

```

```

1706 | | | | | ((mode = nextMode), resetGame());
1707 | | | | | });
1708 | | | | | },
1709 | | | | | colorButtons.forEach((button) => {
1710 | | | | | | button.addEventListener('click', () => {
1711 | | | | | | | const nextColor = button.dataset.color;
1712 | | | | | | | (function isColor(value) {
1713 | | | | | | | | return 'w' === value || 'b' === value;
1714 | | | | | | | })(nextColor) &&
1715 | | | | | | | nextColor !== humanColor &&
1716 | | | | | | | ((humanColor = nextColor), resetGame());
1717 | | | | | | });
1718 | | | | | | },
1719 | | | | | presetButtons.forEach((button) => {
1720 | | | | | | button.addEventListener('click', () => {
1721 | | | | | | | const preset = button.dataset.preset;
1722 | | | | | | | (function isAiPreset(value) {
1723 | | | | | | | | return 'easy' === value || 'normal' === value || 'hard' === value;
1724 | | | | | | | })(preset) && updateAiSettings({ ... AI_PRESETS[preset] });
1725 | | | | | | | });
1726 | | | | | | | },
1727 | | | | | clockPresetButtons.forEach((button) => {
1728 | | | | | | button.addEventListener('click', () => {
1729 | | | | | | | const preset = button.dataset.clockPreset;
1730 | | | | | | | (function isClockPreset(value) {
1731 | | | | | | | | return (
1732 | | | | | | | | | '1/1' === value ||
1733 | | | | | | | | | '3/2' === value ||
1734 | | | | | | | | | '5/3' === value ||
1735 | | | | | | | | | '10/5' === value ||
1736 | | | | | | | | | '15/10' === value
1737 | | | | | | | | );
1738 | | | | | | | | })(preset) &&
1739 | | | | | | | | preset !== clockPreset &&
1740 | | | | | | | | ((clockPreset = preset), resetGame());
1741 | | | | | | | | });
1742 | | | | | | | | },
1743 | | | | | aiDepthInput.addEventListener('input', () => {
1744 | | | | | | var value;
1745 | | | | | | | updateAiSettings({
1746 | | | | | | | | ... aiSettings,
1747 | | | | | | | | maxDepth:
1748 | | | | | | | | | ((value = aiDepthInput.valueAsNumber), Math.min(5, Math.max(1, Math.round(value))));
1749 | | | | | | | | });
1750 | | | | | | | | },
1751 | | | | | aiTimeInput.addEventListener('input', () => {
1752 | | | | | | | updateAiSettings({ ... aiSettings, timeLimitMs: aiTimeInput.valueAsNumber });
1753 | | | | | | | },
1754 | | | | | newGameButton.addEventListener('click', resetGame),
1755 | | | | | undoButton.addEventListener('click', function undoLastTurn() {
1756 | | | | | | (cancelAiMove(),
1757 | | | | | | | (pendingPromotion = null),
1758 | | | | | | | | 0 !== game.history().length
1759 | | | | | | | | ? ('ai' === mode && game.turn() === humanColor ? (game.undo(), game.undo()) : game.undo(),
1760 | | | | | | | | | clearSelection(),
1761 | | | | | | | | | resetClockState(),
1762 | | | | | | | | | restartClockForCurrentTurn(),
1763 | | | | | | | | | render())
1764 | | | | | | | | : render());
1765 | | | | | | | | },
1766 | | | | | promotionChoices.addEventListener('click', (event) => {
1767 | | | | | | | const target = event.target;
1768 | | | | | | | | if (!(target instanceof Element)) return;
1769 | | | | | | | | const button = target.closest('[data-promotion]');
1770 | | | | | | | | promotion = button ? button.dataset.promotion;
1771 | | | | | | | | (function isPromotionPiece(value) {
1772 | | | | | | | | | return 'q' === value || 'r' === value || 'b' === value || 'n' === value;
1773 | | | | | | | | | })(promotion) &&
1774 | | | | | | | | (function finishPromotion(promotion) {
1775 | | | | | | | | | if (!pendingPromotion) return;
1776 | | | | | | | | | if (!pendingPromotion.moves.find((move) => move.promotion === promotion)) return;

```

```

1777 |         const { from: from, to: to } = pendingPromotion;
1778 |         ((pendingPromotion = null), playMove({ from: from, to: to, promotion: promotion }));
1779 |     })(promotion);
1780 |     },
1781 |     resetClockState(),
1782 |     restartClockForCurrentTurn(),
1783 |     render());
1784 | })();
1785 |

```

==== index.html (6983 B / a6648d94f2cc) ====

```

1 | <!doctype html>
2 | <html lang="ko">
3 |   <head>
4 |     <meta charset="UTF-8" />
5 |     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6 |     <title>TS Chess</title>
7 |     <script type="module" crossorigin src="/assets/index-CkN8R2zc.js"></script>
8 |     <link rel="stylesheet" crossorigin href="/assets/index-av5gkzHq.css" />
9 |   </head>
10 |   <body>
11 |     <main id="chess-app" class="chess-app">
12 |       <aside class="control-panel" aria-label="게임 컨트롤">
13 |         <section class="control-group" aria-label="모드">
14 |           <span class="control-label">모드</span>
15 |           <div class="segmented-control" role="group" aria-label="플레이 모드">
16 |             <button class="segmented-button" type="button" data-mode="local">로컬 2인</button>
17 |             <button class="segmented-button" type="button" data-mode="ai">AI 상대</button>
18 |           </div>
19 |         </section>
20 |
21 |         <section id="color-controls" class="control-group" aria-label="진영">
22 |           <span class="control-label">진영</span>
23 |           <div class="segmented-control" role="group" aria-label="인간 진영">
24 |             <button class="segmented-button" type="button" data-color="w">백</button>
25 |             <button class="segmented-button" type="button" data-color="b">흑</button>
26 |           </div>
27 |         </section>
28 |
29 |         <section id="ai-controls" class="control-group ai-settings" aria-label="AI 설정">
30 |           <span class="control-label">AI 설정</span>
31 |           <div
32 |             class="segmented-control segmented-control-three"
33 |             role="group"
34 |             aria-label="AI 프리셋"
35 |           >
36 |             <button class="segmented-button" type="button" data-preset="easy">쉬움</button>
37 |             <button class="segmented-button" type="button" data-preset="normal">보통</button>
38 |             <button class="segmented-button" type="button" data-preset="hard">어려움</button>
39 |           </div>
40 |           <label class="range-control">
41 |             <span>최대 깊이<strong id="ai-depth-value"></strong></span>
42 |             <input id="ai-depth-input" type="range" min="1" max="5" step="1" />
43 |           </label>
44 |           <label class="range-control">
45 |             <span>시간 제한<strong id="ai-time-value"></strong></span>
46 |             <input id="ai-time-input" type="range" min="500" max="5000" step="250" />
47 |           </label>
48 |         </section>
49 |
50 |         <section class="control-group clock-settings" aria-label="시간 설정">
51 |           <span class="control-label">시간</span>
52 |           <div class="clock-preset-control" role="group" aria-label="시간 프리셋">
53 |             <button class="segmented-button" type="button" data-clock-preset="1/1">1/1</button>
54 |             <button class="segmented-button" type="button" data-clock-preset="3/2">3/2</button>
55 |             <button class="segmented-button" type="button" data-clock-preset="5/3">5/3</button>
56 |             <button class="segmented-button" type="button" data-clock-preset="10/5">10/5</button>
57 |             <button class="segmented-button" type="button" data-clock-preset="15/10">15/10</button>
58 |           </div>
59 |         </section>
60 |

```

```

61 |         <section class="action-row" aria-label="게임·동작">
62 |             <button id="new-game-button" class="primary-button" type="button">새 게임</button>
63 |             <button id="undo-button" class="ghost-button" type="button">되돌리기</button>
64 |         </section>
65 |     </aside>
66 |
67 |     <section class="board-stage" aria-label="체스판">
68 |         <div id="chess-board" class="chess-board" role="grid" aria-label="체스판"></div>
69 |     </section>
70 |
71 |     <aside class="clock-panel" aria-label="체스·타이머">
72 |         <div id="clock-display" class="clock-display">
73 |             <div id="white-clock" class="clock-card">
74 |                 <span class="clock-side">백</span>
75 |                 <strong id="white-clock-time" class="clock-time"></strong>
76 |             </div>
77 |             <div id="black-clock" class="clock-card">
78 |                 <span class="clock-side">흑</span>
79 |                 <strong id="black-clock-time" class="clock-time"></strong>
80 |             </div>
81 |         </div>
82 |     </aside>
83 |
84 |     <aside class="info-panel" aria-label="게임·정보">
85 |         <section class="status-panel" aria-live="polite">
86 |             <span class="control-label">상태</span>
87 |             <strong id="game-status" class="game-status"></strong>
88 |             <dl class="status-grid">
89 |                 <div>
90 |                     <dt>차례</dt>
91 |                     <dd id="turn-meta"></dd>
92 |                 </div>
93 |                 <div>
94 |                     <dt>모드</dt>
95 |                     <dd id="mode-meta"></dd>
96 |                 </div>
97 |                 <div>
98 |                     <dt>수순</dt>
99 |                     <dd id="move-meta"></dd>
100 |                 </div>
101 |             </dl>
102 |         </section>
103 |
104 |         <section id="ai-candidate-panel" class="ai-candidate-panel" aria-label="AI·정보" hidden>
105 |             <div class="ai-candidate-header">
106 |                 <span class="control-label">AI·정보</span>
107 |                 <span id="ai-candidate-meta" class="ai-candidate-meta"></span>
108 |             </div>
109 |             <dl class="ai-info-grid">
110 |                 <div>
111 |                     <dt>상태</dt>
112 |                     <dd id="ai-info-status"></dd>
113 |                 </div>
114 |                 <div>
115 |                     <dt>난이도</dt>
116 |                     <dd id="ai-info-preset"></dd>
117 |                 </div>
118 |                 <div>
119 |                     <dt>인간</dt>
120 |                     <dd id="ai-info-human"></dd>
121 |                 </div>
122 |                 <div>
123 |                     <dt>depth</dt>
124 |                     <dd id="ai-info-depth"></dd>
125 |                 </div>
126 |                 <div>
127 |                     <dt>nodes</dt>
128 |                     <dd id="ai-info-nodes"></dd>
129 |                 </div>
130 |                 <div>
131 |                     <dt>score</dt>

```



```

132 |         <dd id="ai-info-score"></dd>
133 |     </div>
134 | </dl>
135 | <div class="ai-candidate-table" role="table" aria-label="AI 후보 수 평가">
136 |     <div class="ai-candidate-row ai-candidate-head" role="row">
137 |         <span role="columnheader">SAN</span>
138 |         <span role="columnheader">cp</span>
139 |         <span role="columnheader">weight</span>
140 |         <span role="columnheader">prob</span>
141 |     </div>
142 |     <div id="ai-candidate-list" class="ai-candidate-list"></div>
143 | </div>
144 | </section>
145 |
146 | <section class="history-panel" aria-label="기보">
147 |     <div class="history-header">
148 |         <span class="control-label">기보</span>
149 |     </div>
150 |     <div id="move-list" class="move-list"></div>
151 | </section>
152 | </aside>
153 |
154 | <div id="promotion-overlay" class="promotion-overlay" hidden>
155 |     <div
156 |         class="promotion-dialog"
157 |         role="dialog"
158 |         aria-modal="true"
159 |         aria-labelledby="promotion-title"
160 |     >
161 |         <h2 id="promotion-title">승격</h2>
162 |         <div
163 |             id="promotion-choices"
164 |             class="promotion-choices"
165 |             role="group"
166 |             aria-label="승격 기물"
167 |         ></div>
168 |     </div>
169 | </div>
170 | </main>
171 | </body>
172 | </html>
173 |

```